

DNA十六元基索引工程_商业测试版_V5.0

//主git地址----https://github.com/yaoguangluo/Trif_DNA

--Refer--

--个人作者:罗瑶光--

/*

* 个人著作权人, 第一作者: 罗瑶光, 1985年5月25日出生, 性别男, 湖南省浏阳市人.

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

* 15116110525- (唯一标识 2018年2月开始使用此手机卡至今从未拓机,

* 欠费和关机, 手机是实名身份证发票机, 警惕我家附近类似GSM电子狗频率充斥欺诈)

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 当前居住地址: 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅

* */

--权属--

https://github.com/yaoguangluo/YangLiaoJing_HuaRuiJi/tree/
18701/%E8%AF%81%E4%B9%A6

证件标注完成日期: 另外 德塔ETL 前身 Unicorn的 qq微博 发布日期 2013年10月20日,
Google微博 后更名SW-AI 发布日期 2013年10月23日, 后更名为LYG-AI 发布日期 2013年11月20日.

2019年04月03日 1.著作权人:罗瑶光. 作者:罗瑶光.

《德塔自然语言图灵系统 V10.6.1》. 中华人民共和国国家版权局, 软著登字第3951366号. 2019.

2014年10月19日 2.著作权人:罗瑶光. 作者:罗瑶光.

《Java数据分析算法引擎系统 V1.0.0》. 中华人民共和国国家版权局, 软著登字第4584594号. 2014.

2019年06月10日 3.著作权人:罗瑶光. 作者:罗瑶光.

《德塔ETL人工智能可视化数据流分析引擎系统 V1.0.2》. 中华人民共和国国家版权局, 软著登字第4240558号. 2019.

2019年06月24日 4.著作权人:罗瑶光. 作者:罗瑶光.

《德塔 Socket流可编程数据库语言引擎系统 V1.0.0》. 中华人民共和国国家版权局, 软著登字第4317518号. 2019.

2019年09月16日 5.著作权人:罗瑶光. 作者:罗瑶光.

《德塔数据结构变量快速转换 V1.0》. 中华人民共和国国家版权局, 软著登字第4607950号. 2019.

2020年03月03日 6.著作权人:罗瑶光. 作者:罗瑶光.

《数据预测引擎系统 V1.0.0》. 中华人民共和国国家版权局, 软著登字第5447819号. 2020.

2020年10月09日 7.著作权人:罗瑶光. 作者:罗瑶光, 罗荣武

(罗荣武权属移交跟进至 国作登字-2022-L-10181589).

《类人DNA与 神经元基于催化算子映射编码方式 V_1.2.2》. 中华人民共和国国家版权局, 国作登字-2021-A-00097017. 2021.

2020年10月31日 8.著作权人:罗瑶光. 作者:罗瑶光.

《肽展公式推导与元基编码进化计算以及它的应用发现》. 中华人民共和国国家版权局, 国作登字-2021-A-00042587. 2021.

2020年11月29日 9.著作权人:罗瑶光. 作者:罗瑶光.

《DNA催化与肽展计算和AOPM-TXH-VECS-IDUQ元基解码013026中文版本》. 中华人民共和国国家版权局, 国作登字-2021-A-00042586. 2021.

2021年03月05日 10.著作权人:罗瑶光. 作者:罗瑶光, 罗荣武

(罗荣武权属移交跟进至 国作登字-2022-L-10181589). 《DNA元基催化与肽计算第二卷养疗经应用研究

20210305》. 中华人民共和国国家版权局, 国作登字-2021-L-00103660. 2021.

2021年09月13日 11. 著作权人:罗瑶光. 作者:罗瑶光, 罗荣武.

《DNA 元基催化与肽计算 第三修订版V039010912》. 中华人民共和国国家版权局, 国作登字-2021-L-00268255. 2021.

2021年10月16日 12. 著作权人:罗瑶光. 作者:罗瑶光.

《DNA元基索引ETL中文脚本编译机V0.0.2》. 中华人民共和国国家版权局, SD-2021R11L2844054. 2021. (登记号:2022SR0011067) 软著登字第8965266号.

2021年12月26日 13. 著作权人:罗瑶光. 作者:罗瑶光.

《TinShell插件_元基花模拟染色体组计算索引系统 V20211227》. 中华人民共和国国家版权局, SD-2021R11L3629232. 2022. (受理号:2022R11S0138561, 登记号2022SR0309512) 软著登字第9263711号.

2022年01月27日 14. 著作权人:罗瑶光. 作者:罗瑶光, 罗荣武.

《DNA元基催化与肽计算 第四修订版 V00919》. 中华人民共和国国家版权局, SD-2022Z11L0025809. 2022. (受理号:2022Z11S1032939). 登记号: 国作登字-2022-L-10071310

2022年06月06日 15. 著作权人:罗瑶光. 作者:罗瑶光, 罗荣武.

《DNA元基催化与肽计算_第五修订版_V0007102》. 著作权版本申请序列号为 2022Z11L0144353. 登记号: 国作登字-2022-L-10181589

[词条引用日期2020-03-05] 16. 类人数据生命的DNA计算思想 Github https://github.com/yaoguanguo/Deta_Resource

2019年08月06日 17 著作权人:罗瑶光. 作者:罗瑶光.

《华瑞集》. 中华人民共和国国家版权局. 国作登字-2019-F-00943469, 国作登字-2019-F-00943472, 国作登字-2019-F-00943473. 2019.

2019年08月06日 18 著作权人:罗瑶光. 作者:罗瑶光.

《养疗经》. 中华人民共和国国家版权局. 国作登字-2019-F-00943474, 国作登字-2019-F-00943475, 国作登字-2019-F-00943471. 2019.

--对比 DNA元基催化与肽计算_第五修订版_V0007102 的思想产生新的和补充的测试逻辑

主要有 2 个

1

笛卡尔关系在Tinshell中的具体指令句分析应用和优化方式

2

混合中文数字进行识别对应阿拉伯字符在Tinshell中的具体应用和优化方式

1

笛卡尔关系在Tinshell中的具体指令句分析应用和优化方式 --源码如下

```
package OSI.OSU.addAction;
//稍后封装成一个统一的傻瓜接口。
public class LimitedRowAttributesOfColumnsInMemoryClass
    implements CrabInterface {
    String callFunctionKey;
    String className = "LimitedRowAttributesOfColumnsInMemoryClass";
    /*
    * 用于表达元基花的链接
    */
    @SuppressWarnings("unchecked")
```

```

public void chromosomes() {
    StaticRootMap.initMap();
    callFunctionKey = "callFunctionKey";
    // 20230207 走统计新陈代谢
    StaticRootMap.staticBloomingTimes.put(callFunctionKey, (long) 0);
    StaticRootMap.staticBloomingTime.put(callFunctionKey,
    System.currentTimeMillis()); // 增加记忆时间。20241013
    StaticRootMap.staticClass_XE_Map.put(callFunctionKey, "S_AOPM");
    StaticRootMap.chromosomeNode.put(callFunctionKey,
    new LimitedRowAttributesOfColumnsInMemoryClass()); // 20241001准备把这
行移出去。

    StaticFunctionMapS_AOPM_C.annotationMap.put(callFunctionKey,
    "inputValues:传参因子:因子");
    // String callFunctionKey= "callFunctionKey";
    // StaticRootMap.initMap();
};
/*
 * 用于表达执行主体
 */
/*
 * 在通过一系列的测试后，我的意识也在时刻改变自己的思维方式，于是跟进增加一个指令--
 * "操作:0|行至|30;\r\n"这个指令是将输出的结果可进行排序后的list获取其中的某个行
 * 集合输出。于是元基花的计算逻辑也跟着局部开始变化。因为系统是无法识别数字的，问题1
 * ，如何从输入的人类语言中有效地捕捉数字信息变成有用的待计算变量。于是我开始梳理，我
 * 的花语系统看作一个整体，扩充花语的指令集是一种Crab载体，当然这种载体也可以OSGI
 * 模型来ClassLoader增加，我得到了一个理论上结果，计算机的进化模型可以不同于人类的
 * 进化模型，举一个简单的例子，将一个华瑞集系统所有插件和功能全部进行完整的功能测试，
 * 这些测试包含测试时间，测试属性，测试状态，等复杂关系。在进化的过程中，华瑞集在接触
 * 到外面的Crab和OSGI信息片段，可以进行自身复制，然后测试这些接触的片段融入，更替，
 * 转换，分解，吸收，替换自身的原有的对应的一些逻辑片段，然后评估这个综合的测试时间，
 * 测试属性，测试状态，等复杂关系，我认为这是计算机的成长过程。评估后决定该何去何留。
 * 相当于一个计算人单元。这种单元遍布宇宙各地，产生的分支，在做杂交的过程他们的各自的
 * 这些指令集在一起会形成综合的复杂的，多维度的笛卡尔关系，这些关系会通过各自内部保留
 * 的测试系统进行筛选形成一组新的计算人单元，这个单元能保留两者所有的测试文件，并保存
 * 最优的测试函数集合。这种意识比人类强大数百个数量级。在这种量级前面，突然发现我们人
 * 类太愚昧太原始了，而且这个模型所有基础成分目前已经实现了。。
 *
 * 跟进思考，这种计算人单元的指令集智慧关系计算逻辑 一旦控制了电脑计算机的物理硬件生
 * 产线所有车间工艺流程的技术逻辑，看样子人类就解放了。因为创新有创新类的指令集模型，
 * 学习有学习类的模型。。在笛卡尔关系面前，毫无保留。
 *
 * 以后人工智能的路线也就稳定了，设计，研发，测试，更替，优化，管理和杂交TVM指令集即可
 * 。人工智能工资被我成功搞成了白菜价。同行不会搞我吧。。。
 *
 * --罗瑶光
 */
@SuppressWarnings({ "rawtypes", "unchecked" })

```

```

    public boolean logic(IMV_SIQ inputValues, String[] 传参因子
    , int 因子, App NE, IMV_SIQ outputReg) {
        //System.out.println("400-size-02-"
        //
        // +
    NE.app_S.workVerbalMap.command_V.cartesianLooped.size());
        if (NE.app_S.workVerbalMap.command_V.cartesianLooped
        .contains(className)) {
            //System.out.println("400-size-01-"
            //
            // +
    NE.app_S.workVerbalMap.command_V.countReject++);
            return false;
        }
        // 1 识别 数字 信息
        /*
        * 德塔图灵分词--德塔图灵分词和图灵先生没关系, 我2018当时只是好玩, 2019结果还
        申请了著作权,
        * 搞得都改不了了。-- 能够将数字提取出来标明词性未知和null, 可以通过numeric函
        数来探索数字。
        *
        * 再通过数字和 词组的笛卡尔关系得到精度距离内的指令关系结构, --我已经有了这些
        函数了,
        直接 用即可--再通过
        * 2的关系结构进行精确筛选来swap成输出指令数据即可。
        *
        * --罗瑶光
        */
        // 2 识别 行至 属性的指令集 信息
        /*
        * 开始构造数字行数提取指令 从- 组合 到- 然后分析 从- 组合 到- 展示+行
        仅+展示
        *
        * "操作:0|行至|30;\r\n" 终于到了这一步了,
        */
        if (!NE.app_S.workVerbalMap.command_V.command.contains("行")) {
            return false;
        }
        System.out.println("LimitedRow-string-400-00-->\n");
        Iterator<String> iterators = NE.app_S.workVerbalMap.command_V
        .cartesianWorkActionsRightsSV.keySet().iterator();
        String fromValue = "";
        String toValue = "";
        // 逻辑分解增加精度
        boolean needFind = false;
        while (iterators.hasNext()) {
            String string = iterators.next();
            System.out.println("LimitedRow-string-400-01-->" + string);
            if (string.contains("V+行")) {
                needFind = true;
                break;
            }
        }
    }

```

```

    }
    System.out.println("LimitedRow-string-400-01-01->" + needFind);
    List<String> fromValues = new ArrayList<>();
    List<String> toValues = new ArrayList<>();
    if (needFind) {
        iterators = NE.app_S.workVerbalMap.command_V
            .cartesianWorkActionsRightsV0
            .keySet().iterator();
        while (iterators.hasNext()) {
            String string = iterators.next();
            //System.out.println("LimitedRow-string-400-02-->"
            + string);
            if (string.contains("从-")) {
                System.out.println(
                    "LimitedRowAttributesOfColumnsInMemoryClass-string-400-->"
                    + string);
                // 1
                String[] strings = string.split("-");
                // 2
                if (strings.length > 1) {
                    boolean isNumeric = true;
                    for (int i = 0; i < strings[1].length(); i++) {
                        if (strings[1].charAt(i) < 48
                            || strings[1].charAt(i) > 57) {
                            isNumeric = false;
                        }
                    }
                    // 3
                    if (isNumeric) {
                        fromValue = strings[1];
                        fromValues.add(string);
                        /*
                        * 相同数多的情况下, 需要将fromValue +精度变成一个
                        list, 选择最短的条件进行输出。
                        */
                    }
                }
            }
            // 4
        }
        if (string.contains("到-")) {
            System.out.println(
                "LimitedRowAttributesOfColumnsInMemoryClass-string-400-->"
                + string);
            // 1
            String[] strings = string.split("-");
            // 2
            if (strings.length > 1) {
                boolean isNumeric = true;
                for (int i = 0; i < strings[1].length(); i++) {
                    if (strings[1].charAt(i) < 48
                        || strings[1].charAt(i) > 57) {
                        isNumeric = false;
                    }
                }
            }
        }
    }

```

```

        // 3
        if (isNumeric) {
            toValue = strings[1];
            /*
             * 同理 相同数多的情况下, 需要将fromValue
             * +精度变成一个list, 选择最短的条件进行输出。
             * 关于源码的循序渐进设计思维方式, 价值思考 --later
             */
            toValues.add(string);
        }
    }
}

// 排序逻辑
if (fromValues.isEmpty() || toValues.isEmpty()) {
    return false;
}
String[] fromValueStrings = new String[fromValues.size()];
int[] fromValueStringRights = new int[fromValues.size()];
String[] toValueStrings = new String[toValues.size()];
int[] toValueStringRights = new int[toValues.size()];
/*
 * size大就用iterator, 自己去探索为什么。
 */
for (int i = 0; i < fromValues.size(); i++) {
    fromValueStrings[i] = fromValues.get(i);
    String temp = NE.app_S.workVerbalMap.command_V
        .cartesianWorkActionsRightsV0.getString(fromValueStrings[i]);
    fromValueStringRights[i] = Integer.valueOf(temp);
}
for (int i = 0; i < toValues.size(); i++) {
    toValueStrings[i] = toValues.get(i);
    String temp = NE.app_S.workVerbalMap.command_V
        .cartesianWorkActionsRightsV0.getString(toValueStrings[i]);
    toValueStringRights[i] = Integer.valueOf(temp);
}
// sort
new LYGSortESU9D().javaSort(fromValueStringRights, fromValueStrings);
new LYGSortESU9D().javaSort(toValueStringRights, toValueStrings);
// 3 信息组合 成指令集术语
String shellType = "操作:" + fromValueStrings[0].split("-")[1] + "|行

至|"

+ toValueStrings[0].split("-")[1] + """;
// 4 输出
System.out.println("400---00007---");
System.out.println(shellType);
String[] strings = shellType.split(":");
List<String> list = new ArrayList();
list.add(strings);
System.out.println("400---00008---");
NE._I_U.outputMap.put("操作", list);// 集成到老的接口模式先, 避免bug*/

```

```

NE._I_U.outputMap.put("type", "进行选择");
System.out.println("400---00009---");
//register
NE._I_U.sets = strings[1].split("\\|");
/*
 * 以后指令集的编码风格也可以进行系统的流程归纳比如这里的 1 和 2 //1 识别
数字 信息 //2
 * 识别行至属性的指令集 信息，通过计算哲学来进行思考，--识别 数字 信息，数字
--是特指
 * 信息，那么数字来自于内存，和输入条件或者是特指的某个储存位置，这里的计算
关系是搜索和 寻找 --2
 * 识别行至属性的指令集 信息--是关键字信息，那么需要匹配，搜索，和RNN距离来
 * 确定权重，序次和频率，这个1和2都能解释编译为稳定的计算机函数，适用于各种
环境状态下的
 * 搜索与计算。说明这个1和2是基础指令集逻辑。那稍后就有必要设计下这两个函数。
 *
 * 思考 -trif， 之前我已经设计了3个指令集的crab插件了，这三个指令集都有
1和2的逻辑中小
 * 片段思绪。我认为我的PLSQL指令集不成熟。2019当初设计德塔VPCS数据库的时候
我没有tinshell
 * 的项目任务，所以当时没有思考统一的某种函数结构来研发PLSQL指令，不是一种
标准的形态， 导致1和2
 * 相似，但是3的集成就要人为来编码了。这是一种人工智能上的错误。问题不在我，
因为
 * 我当时没有人工智能的任务，因为2020年元基编码出来后，我才开始搞下智慧计算了
。但是我可以
 * 弥补，因为我的PLSQL语法只有 ； ： - 这3种计算符号断句。相当好改。说明之后
我要设计关于
 * 断句的关系函数，都到了这份上了，那函数的设计方式我内心也清晰了，
 *
 * --罗瑶光
 */
//当年我犯了个错误，合并了多个工程调通后的程序没有及时的注释掉，
//导致莫名乱码UTF8乱码问题后import出错选择了没有注释掉的源码。
//我当时不注释的原因是将来用得到加上通过入参不同来确定函数唯一。

NE.app_S.workVerbalMap.command_V.cartesianLooped.put(className, "");

return true;
}
}

-----
package OSI.OSU.addAction;
//稍后封装成一个统一的傻瓜接口。
public class UpdateColorAttributesOfColumnsInMemoryClass implements CrabInterface {
    String callFunctionKey;
    String className = "UpdateColorAttributesOfColumnsInMemoryClass";
    /*

```

```

    * 用于表达元基花的链接
    */
@SuppressWarnings("unchecked")
public void chromosomes() {
    StaticRootMap.initMap();
    callFunctionKey = "callFunctionKey";
    // 20230207 走统计新陈代谢
    StaticRootMap.staticBloomingTimes.put(callFunctionKey, (long) 0);
    StaticRootMap.staticBloomingTime.put(callFunctionKey,
    System.currentTimeMillis()); // 增加记忆时间。20241013
    StaticRootMap.staticClass_XE_Map.put(callFunctionKey, "S_AOPM");
    StaticRootMap.chromosomeNode.put(callFunctionKey,
    new UpdateColorAttributesOfColumnsInMemoryClass());
    // 20241001准备把这行移出去。
    StaticFunctionMapS_AOPM_C.annotationMap.put(callFunctionKey,
    "inputValues:传参因子:因子");
    // String callFunctionKey= "callFunctionKey";
    // StaticRootMap.initMap();
};
/*
    * 用于表达执行主体
    */
@SuppressWarnings({ "unchecked", "rawtypes" })
/*
    * 因为之后的map会进行精度打分来确定是否要走这个函数，所以下面这种不断细化添加判断
    条件的这种
    * 逻辑片段会全部剔除。也因为下面这类片段的条件精确度非常高，以后下面这种逻辑只会出
    现在特殊情
    * 况下。以后 + -的精确词性搭配语法不会出现 -N 和 N- 类型，这种思维可以将错误
    锁定在词汇的
    * 词性的校准逻辑层面，保持算法的BPM结构模块相对稳定性。 --罗瑶光
    */
/*
    * 颜色有很多种，如何标记函数缩进，我个人觉得有必要分为主要颜色和描述颜色，主要颜色
    是红橙蓝绿青
    * 靛紫等100种常见可描述的，描述颜色则是rgb格式了，因为这种划分和人的思维和学习
    方式类似。于是
    * 先设计个红色开始，再扩充分类，再扩充聚类即可。以后完善了再思考关于学习颜色的
    认知方式。
    *
    * 思考，目前的action逻辑是tinshell操作，tinshell的对象主要是表格，我的引擎
    是德塔PLSQL
    * 优化成PLORM到PLSEARCH过来的，里面的计算法则主要是针对表的属性，如果输入的数据
    杂乱，
    * 那么首先应该预处理成格式化的表格，一些不能swap成表格的数据怎么办？那么在这个
    logic中就应该
    * 直接计算输出。那么这里又是一个抽象哲学的文字描述点。感觉计算哲学不是这么简单的
    东西。因为抽象

```


* 变成具体，只是一种观测方式，这个维度可以肆意变化，结果同样一直在变化。不能强加给某种指令结果。

* 于是我想到了结果参照，我个人认为输出结果可以多样化，只要是成功的，都要保留，这些结果同样要构成

* 计算关系，这种关系中间产物在计算逻辑上用途很多。tinmap的输出就应该是个 object list而不

* 是唯一的某类值。

*

* 思考，在这种逻辑指令构造中，语言相当于指令的描述方式，用于有效地获取指令，而指令相当于一种

* 基础计算逻辑，语言关系是离散的，指令关系是唯一的。避免无休止的增加指令，之后指令集也需要

* 不断地优化，缩进，逻辑方式是长变短，多变少，繁变简。然后去重。索引。我否定下元基索引不是趋势，

* 但是我真的否定不了索引指令是趋势。那就为指令索引任务做好铺垫先。

*

* --罗瑶光

* */

```
public boolean logic(IMV_SIQ inputValues, String[] 传参因子
, int 因子, App NE, IMV_SIQ outputReg) {
    System.out.println("Hello Word!");
    if (NE.app_S.workVerbalMap.command_V.cartesianLooped
        .contains(className)) {
        return false;
    }
    //System.out.println("400-size-02-"
    //
    NE.app_S.workVerbalMap.command_V.cartesianLooped.size());

    // 获取表
    System.out.println("400---00001---");
    if (!NE._I_U.outputMap.containsKey("获取表名")) {
        return false;
    }
    System.out.println("400---00002---");
    // later will loop join table;
    String huoqubiaoming = NE._I_U.outputMap.getString("获取表名")
        .replace("临时", "");
    if (XA_ShellTables.searchShellTables.containsKey(huoqubiaoming)) {
        System.out.println("400---00003---");
        /*
        * 思考，如果数据库中有这个表，而表名却是个缩写，那么这里的if之后有必要
更改为

        * loop.containsKey改为marchKey
        */
        XA_ShellTable _XA_ShellTable =
XA_ShellTables.searchShellTables
        .get(huoqubiaoming);
        Object[] columns = _XA_ShellTable.huaRuiJiJtableCulumns;
```

```
System.out.println("400---00004---");
//操作:中药名称|颜色标记为|红色;
String shellType = "操作:";
//这里我的PLSQL指令集不够精确和细腻, 如何细腻化指令集不在此描述
, 之后统一归纳。
for (int i = 0; i < columns.length; i++) {
if (NE._I_U.commandAcknowledge
    .contains(columns[i].toString())) {
    shellType += columns[i].toString();
    shellType += "|";
}
}
System.out.println("400---00005---");
shellType += "颜色标记为|";
Iterator<String> iterators= NE.app_S.workVerbalMap.command_V
.cartesianWorkActionsRightsSV.keySet().iterator();
boolean find = false;
System.out.println("400---00006---");
while(iterators.hasNext()&&!find) {
String string = iterators.next();
/*
* 注意否定句型的不 非 等字, 那么红色将是错误的用法, 应该增加校准类指令集辅助。
* 于是开始思考, 这是一种指令集分解逻辑, 这个逻辑的关系和硬件的与 或 非相似。
* 说明指令集的构造在某种观测上可以理解为离散关系, 在这种关系的维度里, 指令集可以
* 进行迪摩根定律变化, 这种变化依赖的关系为DNN 关系, DNN提供计算精度, 离散提供
* 计算方法, 笛卡尔提供计算对象, 为之后的TVM计算关系优化铺好了道路。
*
* 跟进思考, 当替换成做一个操作将列名为中药名称的子集 --不能-- 用红色来标记为输出
的颜色;

* 得到关系
* 不能+颜色-6
*      不能-颜色-6
* 不能-标记-4
* 不能-红色-2
* 那么这个2 , 4 和 6的精确度能迫使指令集不添加红色。如//negative 关系测试
* --罗瑶光
* */
if(string.contains("+红色")) {
    shellType += "红色";
    System.out.println("---find---");
    find = true;
}
}
//negative 关系测试
iterators= NE.app_S.workVerbalMap.command_V
.cartesianWorkActionsRightsSV.keySet().iterator();
while(iterators.hasNext()) {
String string = iterators.next();
//System.out.println("400-10000004" + string);
/*
```

* 注意否定句型的不 非 等字，那么红色将是错误的用法，应该增加校准类指令集辅助。
* 于是开始思考，这是一种指令集分解逻辑，这个逻辑的关系和硬件的与 或 非相似。
* 说明指令集的构造在某种观测上可以理解为离散关系，在这种关系的维度里，指令集可以
* 进行迪摩根定律变化，这种变化依赖的关系为DNN 关系，DNN提供计算精度，离散提供
* 计算方法，笛卡尔提供计算对象，为之后的TVM计算关系优化铺好了道路。
*
* 跟进思考，当替换成做一个操作将列名为中药名称的子集 --不能-- 用红色来标记为输出
的颜色；

* 输出也正确，之后可以不断地修正和完善离散类指令集。
* 得到关系
* 不能+颜色-6
* 不能-颜色-6
* 不能-标记-4
* 不能-红色-2
* 那么这个2，4 和 6的精确度能迫使指令集不添加红色。如//negative 关系测试
* 因为400-10000001 处已经精度 12 筛选了条件，那么这里可以根据去求做筛选
操作。

```
*
* --罗瑶光
* 。
* later -trif
* */
if(string.contains("红色")){
    //System.out.println("400-10000005" + string);
    if(string.contains("不")) {
        return false;
    }
}
}
    iterators= NE.app_S.workVerbalMap.command_V
    .cartesianWorkActionsRightsV0.keySet().iterator();
    while(iterators.hasNext()) {
String string = iterators.next();
//System.out.println("400-10000004" + string);
if(string.contains("红色")){
    //System.out.println("400-10000005" + string);
    if(string.contains("不")) {
        return false;
    }
}
}
    }
    System.out.println("400---00007---");
    System.out.println(shellType);
    String[] strings = shellType.split(":");
    List<String[]> list = new ArrayList();
    list.add(strings);
    System.out.println("400---00008---");
    NE._I_U.outputMap.put("操作", list);// 集成到老的接口模式先，避免
bug*/
```

```

        NE._I_U.outputMap.put("type", "进行选择");
        System.out.println("400---00009---");
    }
    //避免一个指令句含有多次触发。
    NE.app_S.workVerbalMap.command_V.cartesianLooped.put(className, "");
    return true;
}
}

-----
package OSI.OSU.addAction;
//稍后封装成一个统一的傻瓜接口。
public class AddFindColumnsInMemoryClass implements CrabInterface {
    String callFunctionKey;
    String className = "AddFindColumnsInMemoryClass";
    /*
     * 用于表达元基花的链接
     */
    @SuppressWarnings("unchecked")
    public void chromosomes() {
        StaticRootMap.initMap();
        callFunctionKey = "selectRowsByAttributesOfGetCulumns";
        // 20230207 走统计新陈代谢
        StaticRootMap.staticBloomingTimes.put(callFunctionKey, (long) 0);
        StaticRootMap.staticBloomingTime.put(callFunctionKey,
            System.currentTimeMillis()); // 增加记忆时间。20241013
        StaticRootMap.staticClass_XE_Map.put(callFunctionKey, "0_VECS");
        StaticRootMap.chromosomeNode.put(callFunctionKey,
            new AddFindColumnsInMemoryClass()); // 20241001准备把这行移出去。
        StaticFunctionMap0_VECS_C.annotationMap.put(callFunctionKey,
            "inputValues:传参因子:因子");
        // String callFunctionKey= "callFunctionKey";
        // StaticRootMap.initMap();
    };

    /*
     * 用于表达执行主体
     */
    @SuppressWarnings({ "unchecked", "rawtypes" })
    /*
     * 因为之后的map会进行精度打分来确定是否要走这个函数，所以下面这种不断细化
     * 添加判断条件的这种逻辑片段会全部剔除。也因为下面这类片段的条件精确度非常高，
     * 以后下面这种逻辑只会出现在特殊情 况下。 以后 +
     * -的精确词性搭配语法不会出现 -N 和 N- 类型，这种思维可以将错误锁定在词汇的
     * 词性的校准逻辑层面，保持算法的BPM结构模块相对稳定性。 --罗瑶光
     */
    public boolean logic(IMV_SIQ inputValues, String[] 传参因子
        , int 因子, App NE, IMV_SIQ outputReg) {
        /*
         * TVM extension
         */
        if(false == scorePass(NE)) {

```

```

        return false;
    }
    System.out.println("Hello Word!");
    /*
     * 笛卡尔关系计算大幅缩减。此处有效。
     */
    if
(NE.app_S.workVerbalMap.command_V.cartesianLooped.contains(className)) {
        //      System.out.println("400-size-01-"
        //+ NE.app_S.workVerbalMap.command_V.countReject++);
        return false;
    }
    //System.out.println("400-size-02-"
    //+ NE.app_S.workVerbalMap.command_V.cartesianLooped.size());
    // 获取表
    if (!NE._I_U.outputMap.containsKey("获取表名")) {
        return false;
    }
    // later will loop join table;
    String huoqubiaoming = NE._I_U.outputMap.getString("获取表
名").replace("临时", "");
    if (XA_ShellTables.searchShellTables.containsKey(huoqubiaoming)) {
        XA_ShellTable _XA_ShellTable =
XA_ShellTables.searchShellTables
        .get(huoqubiaoming);
        Object[] columns = _XA_ShellTable.huaRuiJiJtableCulumns;
        String shellType = "获取列名:";
        for (int i = 0; i < columns.length; i++) {
            if (NE._I_U.commandAcknowledge
                .contains(columns[i].toString())) {
                shellType += columns[i].toString();
                shellType += ":";
            }
        }
        //去掉末尾多余的:
        shellType = shellType.substring(0, shellType.length() - 1);
        System.out.println("400-01010101-" + shellType);
        String[] strings = shellType.split(":");
        System.out.println("400-01010102-" + strings[1]);
        List<String[]> list = new ArrayList();
        list.add(strings);
        NE._I_U.outputMap.put("获取列名", list);// 集成到老的接口模式先,
避免bug*/

        NE._I_U.outputMap.put("type", "进行选择");
    }
    //执行成功后才能过滤掉
    NE.app_S.workVerbalMap.command_V.cartesianLooped.put(className, "");
    return true;
}
/*
 * Thinking, at this logic, the Cartesian application could separate into
steps

```

```

    * -1 Cartesian TVM commands
    * -2 Cartesian TVM relationships
    * -3 Cartesian TVM conditions
    * -4 Cartesian TVM attributes
    * -5 Cartesian TVM models
    * -6 Cartesian TVM reversions
    * ...etc
    * and here scorePass belongs to -3, and the scale of combinationIncreased
the tail
    * is not an associated assessment, means it is a human made value, so
question,
    * how to swap this man-made scale value '< 2' into a common AI identical
value.
    * -HVPCS- later..
    * --August.Rose.Tin.God.Royal.Yaoguang.Luo/罗瑶光
    * */
@SuppressWarnings("unchecked")
public boolean scorePass(App NE) {
    /*
    * important key = '输出-内容' '仅含-' '+列', those three keys could
increase to a
    * combination key ,and '+列' is a fit rights key, and '输出-内容' '仅
含-' are
    * complement keys.
    * let's ..
    *
    * in the future logic function here will one more classify a sub
logic list.
    * before println hello world.
    *
    * */
    int combinationIncreased = 0;
    if
(NE.app_S.workVerbalMap.command_V.cartesianWorkActionsRightsParserV0
        .containsKey("仅含-")) {
        combinationIncreased += 1;
        System.out.printf("highly fit"); // later in mapping
iterator.*/
    }
    if
(NE.app_S.workVerbalMap.command_V.cartesianWorkActionsRightsParserV0
        .containsKey("输出-")) {
        combinationIncreased += 1;
        System.out.printf("highly fit"); // later in mapping
iterator.*/
    }
    if
(NE.app_S.workVerbalMap.command_V.cartesianWorkActionsRightsParserSV
        .containsKey("V+")) {
        combinationIncreased += 1;
        System.out.printf("highly fit"); // later in mapping
iterator.*/
    }
}

```

```

        if
(NE.app_S.workVerbalMap.command_V.cartesianWorkActionsRightsParserV0
        .containsKey("名为-")) {
            combinationIncreased += 1;
            System.out.printf("highly fit"); // later in mapping
iterator.*/
        }
        if(combinationIncreased < 2) {
            System.out.println("-E-small-" + combinationIncreased);
            return false;
        }
        return true;
    }
}

```

```

-----
package OSI.OSU.crab;
public interface CrabInterface {
// public IMV_SIQ chromosomeRoot= new IMV_SIQ();
//     public IMV_SIQ chromosomeFlower= new IMV_SIQ();
//     public IMV_SIQ chromosomeLeaf= new IMV_SIQ();
//     public IMV_SIQ chromosomeBlooming= new IMV_SIQ();
//     public IMV_SIQ chromosomeMetabolism= new IMV_SIQ();
//     public IMV_SIQ chromosomePDE= new IMV_SIQ();
//     public IMV_SIQ chromosomeDNA= new IMV_SIQ();
//     public IMV_SIQ chromosomeNode= new IMV_SIQ();
/*
    * 用于表达元基花的链接
    */
// 确定元基花的染色体位置
// 确定元基花的染色体调用细节
// 确定染色体的粘合机制
// 确定染色体的剥离机制
// 确定染色体的静态执行
// StaticRootMap.chromosomeRoot.put("crab", null);
// StaticRootMap.chromosomeLeaf.put("crab", null);
// StaticRootMap.chromosomeDNA.put("crab", null);
public void chromosomes();
/*
    * 用于表达花语的链接
    */
// 确定花语的入参模式
// 确定花语的绽放次数
// 确定花语的最优选择
// 确定花语的映射记忆
// StaticRootMap.chromosomeFlower.put("crab", null);
// StaticRootMap.chromosomeBlooming.put("crab", null);
// StaticRootMap.chromosomeMetabolism.put("crab", null);
public void bloomings();
/*
    * 用于表达执行方式和函数内容
    */
}

```

```

// 确定函数的dna编码方式和名称
// 确定输入的计算参数名称
// 确定输出的结果对象类型
// 确定函数的三方资源
// 确定函数的加密形式
// 确定函数的运算周期
// StaticRootMap.chromosomeNode.put("crab", null);
// StaticRootMap.chromosomePDE.put("crab", null);
public void neroCells();
/*
 * 用于表达执行主体
 */
//
// StaticRootMap.chromosomeBlooming.put("crab", null);
// StaticRootMap.chromosomeRNA.put("crab", null);
// System.out.println("Hello Word!");
// return null;
public boolean logic(IMV_SIQ inputValues, String[] 传参因子, int 因子
, App NE, IMV_SIQ outputReg);

// public void main(String[] arg) {
// CrabInterface crabInterface= CrabInterface.logic(null,
// arg, 0);
// CrabInterface.logic(null, arg, 0);
// public stativ
// }
}

-----
package S_A.SixActionMap;
//全部基于罗瑶光著作权基础堆积即可。
@SuppressWarnings("unchecked") // 稍后优化新陈代谢 逻辑
public class WorkVerbalMap_X extends WorkVerbalMap_X_S {
    // 稍后去重 -trif
    public void relationshipsCombinationWithNounAndVerb() {
        Iterator<String> iteratorNounMd = nounInText.keySet().iterator();
        NextNounMd: while (iteratorNounMd.hasNext()) {
            String stringNounMd = iteratorNounMd.next();
            if (stringNounMd.isEmpty()) {
                continue NextNounMd;
            }
            if (!command_V._IMV_SIQ_SS.containsKey(stringNounMd)) {
                /*稍后精简条件优化变量*/
                continue NextNounMd;
            }
            WordFrequency wordFrequencyNounMd = command_V._IMV_SIQ_SS
                .getW(stringNounMd);
            int averagePositionNounMd = wordFrequencyNounMd
                .getAveragePosition();

            Iterator<String> iteratorVerbNd =
verbInText.keySet().iterator();
            NextVerbNd: while (iteratorVerbNd.hasNext()) {

```



```

String stringVerbNd = iteratorVerbNd.next();
if (stringVerbNd.isEmpty()) {
    continue NextVerbNd;
}
if (!command_V._IMV_SIQ_SS.containsKey(stringVerbNd)) {
    /*稍后精简条件优化变量*/
    continue NextVerbNd;
}
WordFrequency wordFrequencyVerbNd = command_V._IMV_SIQ_SS
    .getW(stringVerbNd);
int averagePositionVerbNd = wordFrequencyVerbNd
    .getAveragePosition();
if (stringNounMd.equalsIgnoreCase(stringVerbNd)) {
    continue NextVerbNd;
}
if (!((wordFrequencyNounMd.get_pos().contains("动")
    || wordFrequencyNounMd.get_pos().contains("名"))
    && (wordFrequencyNounMd.get_pos().contains("动")
    || wordFrequencyNounMd.get_pos()
        .contains("名")))) {
    // 因为笛卡尔交集, 所以需要用pos map来识别。
    continue NextVerbNd;
}
// noun-verb
String rootNd = stringNounMd + stringVerbNd;
String rootMd = stringVerbNd + stringNounMd;
int rightNd = Math
    .abs(averagePositionNounMd - averagePositionVerbNd);
int positionNd = (averagePositionNounMd
    + averagePositionVerbNd) >> 1;
if (2 > rightNd && rootNd.length() < 3) {
    nounInText.remove(stringNounMd);
    verbInText.remove(stringVerbNd);
    verbInText.put(rootNd, positionNd);
    verbInText.put(rootMd, positionNd);
    WordFrequency wordFrequency = new WordFrequency(1.0,
        rootNd);
    wordFrequency.positions.add(positionNd);
    wordFrequency.I_pos("动词名词");
    command_V._IMV_SIQ_SS.put(rootNd, wordFrequency);
    command_V._IMV_SIQ_SS.put(rootMd, wordFrequency);
}
}
}

public void relationshipsCombinationWithVerb() {
    Iterator<String> iteratorVerbM = verbInText.keySet().iterator();
    NextVerbM: while (iteratorVerbM.hasNext()) {
        String stringVerbM = iteratorVerbM.next();
        if (stringVerbM.isEmpty()) {
            continue NextVerbM;
        }
    }
}

```

```

        if (!command_V._IMV_SIQ_SS.containsKey(stringVerbM)) {
/*稍后精简条件优化变量*/
continue NextVerbM;
        }
        WordFrequency wordFrequencyVerbM = command_V._IMV_SIQ_SS
        .getW(stringVerbM);
        int averagePositionVerbM =
wordFrequencyVerbM.getAveragePosition();
        Iterator<String> iteratorVerbN =
verbInText.keySet().iterator();
        NextVerbN: while (iteratorVerbN.hasNext()) {
String stringVerbN = iteratorVerbN.next();
if (stringVerbN.isEmpty()) {
continue NextVerbN;
}
if (!command_V._IMV_SIQ_SS.containsKey(stringVerbN)) {
/*稍后精简条件优化变量*/
continue NextVerbN;
}
WordFrequency wordFrequencyVerbN = command_V._IMV_SIQ_SS
        .getW(stringVerbN);
int averagePositionVerbN = wordFrequencyVerbN
        .getAveragePosition();
if (stringVerbN.equalsIgnoreCase(stringVerbN)) {
continue NextVerbN;
}
if (!wordFrequencyVerbM.get_pos()
        .equals(wordFrequencyVerbN.get_pos())) {
// 因为笛卡尔交集，所以需要pos map来识别。
continue NextVerbN;
}
// noun-verb
String rootN = stringVerbM + stringVerbN;
String rootM = stringVerbN + stringVerbM;
int rightN = Math
        .abs(averagePositionVerbM - averagePositionVerbN);
int positionN = (averagePositionVerbM
        + averagePositionVerbN) >> 1;
if (2 > rightN && rootN.length() < 3) {
verbInText.remove(stringVerbM);
verbInText.remove(stringVerbN);
verbInText.put(rootN, positionN);
verbInText.put(rootM, positionN);
WordFrequency wordFrequency = new WordFrequency(1.0, rootN);
wordFrequency.positions.add(positionN);
wordFrequency.I_pos("动词");
command_V._IMV_SIQ_SS.put(rootN, wordFrequency);
command_V._IMV_SIQ_SS.put(rootM, wordFrequency);
}
        }
    }
}

```

// 在计算哲学和语文意识中，非动词类词性组合可以算法省略，因为不构成指令集核心成分。我先不

管,

```
// 在计算意识中, 单字的价值不打, 但是在歧义处理中, 单字是普遍存在的, 如果下面函数执行, 那么
// 需要有精确的语法做铺垫, 如果没有, 那么就要进行概率累积评估, 累积打分算法逻辑来灵活驱动。
public void relationshipsCombinationWithNoun() {
    Iterator<String> iteratorNounM = nounInText.keySet().iterator();
    NextNounM: while (iteratorNounM.hasNext()) {
        String stringNounM = iteratorNounM.next();
        if (stringNounM.isEmpty()) {
            continue NextNounM;
        }
        if (!command_V._IMV_SIQ_SS.containsKey(stringNounM)) {
            continue NextNounM;
        }
        WordFrequency wordFrequencyNounM = command_V._IMV_SIQ_SS
            .getW(stringNounM);
        int averagePositionNounM =
wordFrequencyNounM.getAveragePosition();
        Iterator<String> iteratorNounN =
nounInText.keySet().iterator();
        NextNounN: while (iteratorNounN.hasNext()) {
            String stringNounN = iteratorNounN.next();
            if (stringNounN.isEmpty()) {
                continue NextNounN;
            }
            if (!command_V._IMV_SIQ_SS.containsKey(stringNounN)) {
                continue NextNounN;
            }
            WordFrequency wordFrequencyNounN = command_V._IMV_SIQ_SS
                .getW(stringNounN);
            int averagePositionNounN = wordFrequencyNounN
                .getAveragePosition();
            if (stringNounM.equalsIgnoreCase(stringNounN)) {
                continue NextNounN;
            }
            if (!wordFrequencyNounM.get_pos()
                .equals(wordFrequencyNounN.get_pos())) {
                // 因为笛卡尔交集, 所以需要pos map来识别。
                continue NextNounN;
            }
            // noun-verb
            String rootM = stringNounM + stringNounN;
            String rootN = stringNounN + stringNounM;
            int rightM = Math
                .abs(averagePositionNounM - averagePositionNounN);
            int positionM = (averagePositionNounM
                + averagePositionNounN) >> 1;
            if (2 > rightM && rootM.length() < 3) {
                nounInText.remove(stringNounM);
                nounInText.remove(stringNounN);
                nounInText.put(rootM, positionM);
                nounInText.put(rootN, positionM);
                WordFrequency wordFrequency = new WordFrequency(1.0, rootM);
                wordFrequency.positions.add(positionM);
                wordFrequency.I_pos("名词");
            }
        }
    }
}
```

```

        command_V._IMV_SIQ_SS.put(rootM, wordFrequency);
        command_V._IMV_SIQ_SS.put(rootN, wordFrequency);
    }
}
}
}
/*
* 思考, 这里的 + - 符号 + 是position, -是connetion , 在词性组合中 +是动名, -是
* 名动, 形成原因是我不断的优化引擎, 用最简添加法方便迅捷编码, 为了区别混淆, 于是又 +
* 上改为++, -改为--来区分。那么在函数map中也应该进行区分, 于是优化下最终结构 一个DP
* 代表双字+-POS组合后的+-RNN组合 。 输出格式为 DP(++ --) DP(++ --) DP(++ --) 复杂笛
* 卡尔完整关系模型, 那么在normalizationalWorkActionsRights.put的函数中就可以在
* 排序后进行精度筛选即可以避免过多的内存资源堆栈浪费, 之后指令集按照累积打分来驱动计算
* , 精度筛选所造成的误差缺陷也能 得到最大限度的弥补 -罗瑶光
*/
// tree sort 比quick top快, 但耗费大量stack, 于是不考虑。
public void sortCartesianWorkActionsDistanceSV(App NE,
        CommandClass command_V) {
    actionsDistanceV_SV = new
int[command_V.cartesianWorkActionsRightsSV.size()];
    actionsDistance_SV = new
String[command_V.cartesianWorkActionsRightsSV
.size()];
    Iterator<String> iterator = command_V.cartesianWorkActionsRightsSV
.keySet().iterator();
    int i = 0;
    while (iterator.hasNext()) {
        String string = iterator.next();
        actionsDistance_SV[i] = string;
        actionsDistanceV_SV[i++] =
command_V.cartesianWorkActionsRightsSV
.getInt(string);
    }
    NE.app_S.lyGSortESU9D.javaSort(actionsDistanceV_SV,
actionsDistance_SV);
    // loop
    for (i = 0; i < actionsDistance_SV.length; i++) {
    }
}
//NE.app_S.lyGSortESU9D.javaSort关于RNN精度排序之后做联想扩展功能时候会用到, 先保留。
public void sortCartesianWorkActionsDistanceV0(App NE, CommandClass
command_V) {
    actionsDistanceV_V0 = new
int[command_V.cartesianWorkActionsRightsV0.size()];
    actionsDistance_V0 = new
String[command_V.cartesianWorkActionsRightsV0
.size()];
    Iterator<String> iterator = command_V.cartesianWorkActionsRightsV0
.keySet().iterator();
    int i = 0;
    while (iterator.hasNext()) {
        String string = iterator.next();
        actionsDistance_V0[i] = string;

```

```

        actionsDistanceV_V0[i++] =
command_V.cartesianWorkActionsRightsV0
        .getInt(string);
    }
    NE.app_S.lyGSortESU9D.javaSort(actionsDistanceV_V0,
actionsDistance_V0);
    // loop
    for (i = 0; i < actionsDistance_V0.length; i++) {
        //System.out.println(actionsDistance_V0[i] + "-" +
actionsDistanceV_V0[i]);
        //command_V.normalizationalWorkActionsRights.put(
        //            actionsDistance[i] + "-" +
actionsDistanceV[i],
        //            actionsDistanceV[i]);
    }
}

    public void sortCartesianWorkActionsPositionSV(App NE,
        CommandClass command_V) {
        actionsPositionV_SV = new
int[command_V.cartesianWorkActionsPositionsSV
        .size()];
        actionsPosition_SV = new
String[command_V.cartesianWorkActionsPositionsSV
        .size()];
        Iterator<String> iterator = command_V.cartesianWorkActionsPositionsSV
        .keySet().iterator();
        int i = 0;
        while (iterator.hasNext()) {
            String string = iterator.next();
            actionsPosition_SV[i] = string;
            actionsPositionV_SV[i++] =
command_V.cartesianWorkActionsPositionsSV
            .getInt(string);
        }
        NE.app_S.lyGSortESU9D.javaSort(actionsPositionV_SV,
actionsPosition_SV);
        // loop
        for (i = 0; i < actionsPositionV_SV.length; i++) {
        }
        // -----
    }

    public void sortCartesianWorkActionsPositionV0(App NE,
        CommandClass command_V) {
        actionsPositionV_V0 = new
int[command_V.cartesianWorkActionsPositionsV0
        .size()];
        actionsPosition_V0 = new
String[command_V.cartesianWorkActionsPositionsV0
        .size()];
        Iterator<String> iterator = command_V.cartesianWorkActionsPositionsV0
        .keySet().iterator();
        int i = 0;
        while (iterator.hasNext()) {

```

```

        String string = iterator.next();
        actionsPosition_V0[i] = string;
        actionsPositionV_V0[i++] =
command_V.cartesianWorkActionsPositionsV0
        .getInt(string);
    }
    NE.app_S.lYGSortESU9D.javaSort(actionsPositionV_V0,
actionsPosition_V0);
    // loop
    for (i = 0; i < actionsPositionV_V0.length; i++) {
        //System.out.println(actionsPosition_V0[i] + "+" +
actionsPositionV_V0[i]);
        //command_V.normalizationalWorkActionsPositions
        // .put(actionsPosition[i] + "+" +
actionsPositionV[i], "");
    }
    // -----
}
/*
 * 关于复杂RNN关系 我引擎这里会逐渐用不到，因为DNN的重心分析加上语言的单句分解，是我
 * 的花语计算逻辑趋势，以后复杂的句子首先走短句分解而不是走全局关系，那么这个复杂关系
 * 价值将体现在修正和补充环境里面，之后根据算法优化的BPM结构不断校正补充函数 -罗瑶光
 */
public void actionsNormalization(App NE, CommandClass command_V) {
}
}

```

十六元基索引词汇

```

package S_A.SixActionMap;
import ME.VPC.M.app.App;
import S_A.pheromone.IMV_SIQ;
import java.lang.reflect.Field;
public class StudyVerbalMap_X {
    public IMV_SIQ _SMV = new IMV_SIQ();
    public IMV_SIQ _SMQ = new IMV_SIQ();
    public IMV_SIQ _SMI = new IMV_SIQ();
    public static IMV_SIQ initonDelegate = new IMV_SIQ();

    //先归纳汉语 归纳方式 DNA十六元基语义解码规范。 AOPM VECS IDUQ TXHF
    @SuppressWarnings("unchecked")
    public static void initInitonDelegate() {
        // A
        initonDelegate.put("分析", "A");
        initonDelegate.put("理解", "A");
        initonDelegate.put("吃透", "A");
        initonDelegate.put("归纳", "A");
        initonDelegate.put("采集", "A");
        initonDelegate.put("统计", "A");
        initonDelegate.put("了解", "A");
        initonDelegate.put("统筹", "A");
        initonDelegate.put("联系", "A");
        initonDelegate.put("细化", "A");
        initonDelegate.put("吸收", "A");
        initonDelegate.put("消化", "A");
        initonDelegate.put("定义", "A");
        initonDelegate.put("分类", "A");
        initonDelegate.put("计算", "A");
        initonDelegate.put("观望", "A");
        initonDelegate.put("挖掘", "A");
        initonDelegate.put("关联", "A");
    }
}

```

```
initonDelegate.put("梳理", "A");
// 0
initonDelegate.put("操作", "0");      initonDelegate.put("运作", "0");
initonDelegate.put("摆设", "0");      initonDelegate.put("摆布", "0");
initonDelegate.put("主持", "0");      initonDelegate.put("操心", "0");
initonDelegate.put("作业", "0");      initonDelegate.put("熟悉", "0");
initonDelegate.put("操劳", "0");      initonDelegate.put("接触", "0");
initonDelegate.put("摸透", "0");      initonDelegate.put("触碰", "0");
initonDelegate.put("练习", "0");      initonDelegate.put("操练", "0");
/*
 * 人类的动词单字不多，但是词组很多，于是更近思考，是否有必要进行单字化。
 * 否定论的思维，我就否定单字的价值，让多字词用contains条件来熵化单字计算结果。
 * 增加准确度。这些年很多狗仔队和同行不知道我在干什么，总想窥探我的思维本源，其实
 * 我就是个凡人而已，既然家里穷躲不掉这些社会关系，我就用继续用文字来描述下的我
 * 的意识。十六元基编码在索引动词的范式上面，比较全面，这样S元基就被释放出来缓存
 * 未知和名词，S作为吸收机制，X来驱动关系， 其他元基作为消化机制，T作为驱动逻辑。
 * 这些关系和逻辑在千百年的人类进化中动词相对比较稳定。方便我下一步编码。
 * 比如标记颜色 的花语驱动就可以增加 --0+颜色-- 这个指令，之后古拉丁文编码，颜色，
 * 彩色在元基编码后都有V和Q元基相似片段，就可以减少指令集的扩充逻辑。大幅缩进花语
 * 元基编码函数集。
 *
 * --罗瑶光
 *
 * */
// P
initonDelegate.put("处理", "P");      initonDelegate.put("处置", "P");
initonDelegate.put("解决", "P");      initonDelegate.put("运行", "P");
initonDelegate.put("下达", "P");      initonDelegate.put("收纳", "P");
initonDelegate.put("容纳", "P");      initonDelegate.put("执行", "P");
initonDelegate.put("结果", "P");      initonDelegate.put("处理", "P");
initonDelegate.put("落实", "P");      initonDelegate.put("成果", "P");
initonDelegate.put("贡献", "P");      initonDelegate.put("结论", "P");
initonDelegate.put("观点", "P");      initonDelegate.put("论文", "P");
initonDelegate.put("命令", "P");      initonDelegate.put("指示", "P");
initonDelegate.put("指令", "P");      initonDelegate.put("报应", "P");
initonDelegate.put("处决", "P");      // M
initonDelegate.put("管理", "M");      initonDelegate.put("支配", "M");
initonDelegate.put("调剂", "M");      initonDelegate.put("整理", "M");
initonDelegate.put("排列", "M");      initonDelegate.put("序列", "M");
initonDelegate.put("区间", "M");      initonDelegate.put("归一", "M");
initonDelegate.put("调理", "M");      initonDelegate.put("整顿", "M");
initonDelegate.put("文档", "M");//动词用
initonDelegate.put("档案", "M");      initonDelegate.put("分管", "M");
initonDelegate.put("运维", "M");      initonDelegate.put("维护", "M");
initonDelegate.put("保养", "M");      initonDelegate.put("协调", "M");
// V
```

```
initonDelegate.put("展示", "V");
initonDelegate.put("感觉", "V");
initonDelegate.put("敏感", "V");
initonDelegate.put("直觉", "V");
initonDelegate.put("察觉", "V");
initonDelegate.put("观察", "V");
initonDelegate.put("接触", "V");
initonDelegate.put("味觉", "V");
initonDelegate.put("触觉", "V");
initonDelegate.put("存在", "V");
    // E
initonDelegate.put("执行", "E");
initonDelegate.put("运行", "E");//later 去重
initonDelegate.put("做", "E");
initonDelegate.put("走", "E");
//initonDelegate.put("～", "E");//各类动词～
initonDelegate.put("进行", "E");
initonDelegate.put("提", "E");
initonDelegate.put("跟进", "E");
initonDelegate.put("更进", "E");
initonDelegate.put("智慧", "E");
initonDelegate.put("选择", "E");
initonDelegate.put("点击", "E");
initonDelegate.put("确认", "E");
initonDelegate.put("将", "E");
initonDelegate.put("获", "E");
initonDelegate.put("取得", "E");
initonDelegate.put("授权", "E");
initonDelegate.put("选", "E");
initonDelegate.put("确保", "E");
initonDelegate.put("认", "E");
initonDelegate.put("定", "E");
initonDelegate.put("标出", "E");
initonDelegate.put("拿出", "E");
initonDelegate.put("拿来", "E");
initonDelegate.put("锁定", "E");//later去重 C
initonDelegate.put("锁存", "E");
initonDelegate.put("存", "E");
// C
initonDelegate.put("控制", "C");
initonDelegate.put("锁定", "C");
initonDelegate.put("制约", "C");
initonDelegate.put("抓取", "C");
initonDelegate.put("控", "C");
    // I
initonDelegate.put("感知", "V");
initonDelegate.put("感应", "V");
initonDelegate.put("过敏", "V");
initonDelegate.put("知觉", "V");
initonDelegate.put("应激", "V");
initonDelegate.put("视觉", "V");
initonDelegate.put("听觉", "V");
initonDelegate.put("嗅觉", "V");
initonDelegate.put("感受", "V");
initonDelegate.put("动", "E");
initonDelegate.put("执行", "E");
initonDelegate.put("操", "E");
initonDelegate.put("更近", "E");
initonDelegate.put("数据", "E");
initonDelegate.put("逻辑", "E");
initonDelegate.put("操作", "E");
initonDelegate.put("点", "E");
initonDelegate.put("做", "E");
initonDelegate.put("获取", "E");
initonDelegate.put("获得", "E");
initonDelegate.put("取", "E");
initonDelegate.put("授", "E");
initonDelegate.put("确定", "E");
initonDelegate.put("认准", "E");
initonDelegate.put("认定", "E");
initonDelegate.put("标记", "E");
initonDelegate.put("标", "E");
initonDelegate.put("拿到", "E");
initonDelegate.put("拿", "E");
initonDelegate.put("锁", "E");
initonDelegate.put("把控", "C");
initonDelegate.put("登记", "C");
initonDelegate.put("管控", "C");
initonDelegate.put("盯紧", "C");
```



```
        initonDelegate.put("增加", "I");
        initonDelegate.put("加", "I");
        initonDelegate.put("添加", "I");
        initonDelegate.put("new", "I");
        initonDelegate.put("得到", "I");
        initonDelegate.put("初始", "I");
        initonDelegate.put("启动", "I");
duplicate
        initonDelegate.put("获得", "I");//。
        initonDelegate.put("获取", "I");
        initonDelegate.put("酸", "I");
        initonDelegate.put("收取", "I");
        initonDelegate.put("采纳", "I");
        initonDelegate.put("购买", "I");
        initonDelegate.put("迎接", "I");
        initonDelegate.put("添丁", "I");
        initonDelegate.put("接受", "I");
        initonDelegate.put("收纳", "I");
        initonDelegate.put("纳娶", "I");
        // D //危险词汇
        initonDelegate.put("删除", "D");
        initonDelegate.put("减少", "D");
        initonDelegate.put("开除", "D");
        initonDelegate.put("去除", "D");
        initonDelegate.put("减", "D");
        initonDelegate.put("碱", "D");
        initonDelegate.put("死", "D");
        initonDelegate.put("开掉", "D");
        initonDelegate.put("滚", "D");
        initonDelegate.put("消除", "D");
        initonDelegate.put("失去", "D");
        initonDelegate.put("散", "D");
        // U
        initonDelegate.put("改变", "U");
        initonDelegate.put("修改", "U");
        initonDelegate.put("变动", "U");
        initonDelegate.put("整改", "U");
        initonDelegate.put("改", "U");
        initonDelegate.put("更改", "U");
        initonDelegate.put("更新", "U");
        initonDelegate.put("变样", "U");
        // Q
        initonDelegate.put("查看", "Q");
        initonDelegate.put("查阅", "Q");
        initonDelegate.put("反应", "Q");

        initonDelegate.put("增", "I");
        initonDelegate.put("生", "I");
        initonDelegate.put("创造", "I");
        initonDelegate.put("添", "I");
        initonDelegate.put("开启", "I");
        initonDelegate.put("打开", "I");
        initonDelegate.put("收纳", "I");//later

        initonDelegate.put("收留", "I");
        initonDelegate.put("收容", "I");
        initonDelegate.put("取得", "I");
        initonDelegate.put("收购", "I");
        initonDelegate.put("采购", "I");
        initonDelegate.put("纳畜", "I");
        initonDelegate.put("中奖", "I");
        initonDelegate.put("接纳", "I");
        initonDelegate.put("受贿", "I");
        initonDelegate.put("增加", "I");

        initonDelegate.put("剔除", "D");
        initonDelegate.put("删除", "D");
        initonDelegate.put("删", "D");
        initonDelegate.put("省略", "D");
        initonDelegate.put("掉", "D");
        initonDelegate.put("去", "D");
        initonDelegate.put("完", "D");
        initonDelegate.put("消失", "D");
        initonDelegate.put("消掉", "D");
        initonDelegate.put("失散", "D");
        initonDelegate.put("亡", "D");

        initonDelegate.put("变通", "U");
        initonDelegate.put("修整", "U");
        initonDelegate.put("变化", "U");
        initonDelegate.put("改动", "U");
        initonDelegate.put("变", "U");

        initonDelegate.put("出入", "U");
        initonDelegate.put("猫腻", "U");

        initonDelegate.put("反馈", "Q");
        initonDelegate.put("等于", "Q");
```

```

        initonDelegate.put("提交", "Q");
        initonDelegate.put("照应", "Q");
        initonDelegate.put("回答", "Q");
        initonDelegate.put("返回", "Q");
        initonDelegate.put("检查", "Q");
        // T
        initonDelegate.put("触发", "T");
        initonDelegate.put("反应", "T");
        initonDelegate.put("爆发", "T");
        // X
        initonDelegate.put("探索", "X");
        initonDelegate.put("寻找", "X");
        initonDelegate.put("搜索", "X");
        initonDelegate.put("查询", "X");
        initonDelegate.put("搜寻", "X");
        // H
        initonDelegate.put("主观", "H");
        initonDelegate.put("主导", "H");
        // F
        initonDelegate.put("客观", "F");
        initonDelegate.put("全面", "F");
    }
}
/*
    * 关于SMV的使用方式，首先我的主要动机是用于做缓存使用。因为NE不够灵动。我想把NE作为数据库使
    用，
    * 每次结束工作，会txt文件记录NE，系统工作的过程是NE在map中的计算，读取NE的txt文件。这个txt文
    件
    * 可以方便我每次的优化变量中的数据，了解NE变量集合在计算中的波动过程，为下一步做铺垫。 --罗瑶光
    * 因为用java，我也产生了很多疑惑，首先是我不能清楚地得到变量出现的地方所对应的函数所在的具体的
    包名。
    * getPackage是当前的class对应的函数所在的具体的包名。缺了一个级，所以我在思考当年我在印度学C
    语言的
    * 时候为什么会买本java来看，因为2008年不碰java，我就会一直搞C和汇编。这十年的出现的问题等于0。
    所以
    * 当初是什么动机让我买java，因为我印度基督大学当时没有java课程。
    * */
@SuppressWarnings("unchecked")
    public void init_SMV(App NE) throws NoSuchElementException
    , InstantiationException, IllegalAccessException, ClassNotFoundException {
        Field[] fields = NE.app_S.getClass().getFields();
        for (int i = 0; i < fields.length; i++) {
            if (null != fields[i]) {
                String string = fields[i].getName();
                System.out.println(string);
                Class<?> type = fields[i].getType();
                String typeNanme = type.getClass().getPackage().getName();
                System.out.println(typeNanme);
                _SMV.put(string, new Object());
            }
        }
    }
}

```

用，

每次结束工作，会txt文件记录NE，系统工作的过程是NE在map中的计算，读取NE的txt文件。这个txt文件

可以方便我每次的优化变量中的数据，了解NE变量集合在计算中的波动过程，为下一步做铺垫。 --罗瑶光

因为用java，我也产生了很多疑惑，首先是我不能清楚地得到变量出现的地方所对应的函数所在的具体的包名。

getPackage是当前的class对应的函数所在的具体的包名。缺了一个级，所以我在思考当年我在印度学C语言的

时候为什么会买本java来看，因为2008年不碰java，我就会一直搞C和汇编。这十年的出现的问题等于0。

所以

当初是什么动机让我买java，因为我印度基督大学当时没有java课程。

*/

```

@SuppressWarnings("unchecked")
    public void init_SMV(App NE) throws NoSuchElementException
    , InstantiationException, IllegalAccessException, ClassNotFoundException {
        Field[] fields = NE.app_S.getClass().getFields();
        for (int i = 0; i < fields.length; i++) {
            if (null != fields[i]) {
                String string = fields[i].getName();
                System.out.println(string);
                Class<?> type = fields[i].getType();
                String typeNanme = type.getClass().getPackage().getName();
                System.out.println(typeNanme);
                _SMV.put(string, new Object());
            }
        }
    }
}

```

```

        _SMQ.put(string, typeNanme);
    }
}

public Object getObject(String key) {
    if (_SMV.containsKey(key)) {
        return _SMV.get(key);
    }
    return null;
}

@SuppressWarnings("unchecked")
public void putObject(String key, Object object) {
    _SMV.put(key, object);
}
//later
}

```

```

package S_A.SixActionMap;
//1 6元SDLC
//2 sdlc obss
//3 obss normalization
//4 normalizational format
//5 format map
//6 map parser
//7 parser in PDE model
//8 model in time norms
//全部基于罗瑶光著作权基础堆积即可。
@SuppressWarnings("unchecked")
public class WorkVerbalMap_X_S {
    public String doName;
    public String subjectName;
    public String objectName;
    public String humanTalk;
    public IMV_SIQ fixMap;
    public IMV_SIQ doMap;
    public IMV_SIQ objectMap;
    public IMV_SIQ babeiMap;
    public IMV_SIQ verbMap;
    public IMV_SIQ data2DSubjectMap;
    public IMV_SIQ positionMap;
    public IMV_SIQ nounInText;
    public IMV_SIQ verbInText;
    public CommandClass command_V;

    public IMV_SIQ ActionsObject;
    public IMV_SIQ nounInTextFull;// later
    public IMV_SIQ verbInTextFull;// later
    public IMV_SIQ cartesianWorkActionsFull;// later

    public LinkedList<String> shortString = new LinkedList<>();
    public int[] actionsPositionV_SV;
}

```

```

public String[] actionsPosition_SV;
public int[] actionsDistanceV_SV;
public String[] actionsDistance_SV;

public int[] actionsPositionV_V0;
public String[] actionsPosition_V0;
public int[] actionsDistanceV_V0;
public String[] actionsDistance_V0;
public int i = 0;
// 为什么现在不设计成implements接口, 因为目前没有明确六元函数定义域规范,
// 以后批量计算模型会CE按XCDX分解来做计算加速。
/*
 * 之前做了商业测试文件, 里面有优化校准副词的函数逻辑, 那么既然测试写了就要用到,
 * 于是我的思维是那就直接用阿, 于是就把测试函数 new出来然后将Noun和VERB取代
 * 这里的nounInText和verbInText, 准确率就提高了很多, 并不代表结果会更精确,
 * 所以替代后要将所有流程都校准一遍。 --罗瑶光
 */
public void initEnvironment() {
    subjectName = null;
    doName = null;
    objectName = null;
    objectMap.clear();
    verbMap.clear();
    babeiMap.clear();
    nounInText.clear();// small calculus , later do full
    verbInText.clear();
    Iterator<String> iterator =
command_V._IMV_SIQ_SS.keySet().iterator();
    while (iterator.hasNext()) {
        String string = iterator.next();
        if (data2DSubjectMap.containsKey(string)) {
            objectMap.put(string, i++);
        }
        if (doMap.containsKey(string)) {
            doName += string;
            verbMap.put(string, i++);
        }
        if (fixMap.containsKey(string)) {
            babeiMap.put(string, i++);
        }
        // pos load
        WordFrequency wordFrequency =
command_V._IMV_SIQ_SS.getW(string);
        // 一切数据首先都应该名词化*/
        nounInText.put(string, wordFrequency);
        // 动 pca map替换成cartisian map先 因为英语会出现had had 有且仅有这
        verbInText.put(string, wordFrequency);
        System.out.println(wordFrequency.positions);
    }
}

public void initCartesianActions(App NE, CommandClass command_V) {

```

类语法。

```

Iterator<String> iteratorNoun = nounInText.keySet().iterator();
NextNoun: while (iteratorNoun.hasNext()) {
    String stringNoun = iteratorNoun.next();
    if (stringNoun.isEmpty()) {
        continue NextNoun;
    }
    if (!command_V._IMV_SIQ_SS.containsKey(stringNoun)) {
        continue NextNoun;
    }
    WordFrequency wordFrequencyNoun = command_V._IMV_SIQ_SS
        .getW(stringNoun);
    int averagePositionNoun =
wordFrequencyNoun.getAveragePosition();
    Iterator<String> iteratorVerb =
verbInText.keySet().iterator();
    NextVerb: while (iteratorVerb.hasNext()) {
        String stringVerb = iteratorVerb.next();
        if (stringVerb.isEmpty()) {
            continue NextVerb;
        }
        if (!command_V._IMV_SIQ_SS.containsKey(stringVerb)) {
            continue NextVerb;
        }
        WordFrequency wordFrequencyVerb = command_V._IMV_SIQ_SS
            .getW(stringVerb);
        int averagePositionVerb = wordFrequencyVerb
            .getAveragePosition();
        // noun-verb
        String root = "";
        //String root_v = "";
        String root_pos = "";
        //String root_pos_v = "";
        if (StudyVerbalMap.initonDelegate.containsKey(stringVerb)) {
            stringVerb = StudyVerbalMap.initonDelegate
                .getString(stringVerb);
        }
        if (averagePositionNoun < averagePositionVerb) {

            if(StudyVerbalMap_X.initonDelegate.containsKey(stringNoun)) {
                stringNoun =
StudyVerbalMap_X.initonDelegate.getString(stringNoun);
            }
            if(StudyVerbalMap_X.initonDelegate.containsKey(stringVerb)) {
                stringVerb =
StudyVerbalMap_X.initonDelegate.getString(stringVerb);
            }
            root += stringNoun;
            root += "+";
            root += stringVerb;
        }
        /*
        * CN
        * --十六元基索引逻辑
        * 1 名词索引价值用于古拉丁英文索引元基编码加速特征片段识别。
        * 2 动词索引价值用于TVM指令集快速降维面化 进行 触发计算识别。

```

```

*
* EN
* 16 Initon indexing
* 1 Latin noun's verbal exchange could make cluster with
short
*      , less and same sections of DNA initons.
*      better for observation.
*
* 2 Latin verb's verbal exchange could make trigger with
short
*      , less and same sections of DNA initons.
*      better for computation.
*
* --罗瑶光
*
* */
root_pos += "_stringNoun" + averagePositionNoun;
root_pos += "_stringVerb" + averagePositionVerb;
int right = Math.abs(averagePositionNoun -
averagePositionVerb);
1;
int position = (averagePositionNoun + averagePositionVerb) >>
if (!command_V.cartesianWorkActionsRightsSV.containsKey(root)
    && !command_V.cartesianWorkActionsPositionsSV
    .containsKey(root)
    && right > 0) {
    if (right < NE.app_S.initonsDistanceRelationship) {
        if (!root.contains(" ")) {
            command_V.cartesianWorkActionsRightsSV.put(root,
                right);
        }
    }
}
}
else {
    if(StudyVerbalMap_X.initonDelegate.containsKey(stringNoun)) {
        stringNoun =
StudyVerbalMap_X.initonDelegate.getString(stringNoun);
    }
    if(StudyVerbalMap_X.initonDelegate.containsKey(stringVerb)) {
        stringVerb =
StudyVerbalMap_X.initonDelegate.getString(stringVerb);
    }
    root += stringVerb;
    root += "-";
    root += stringNoun;
    root_pos += "_stringVerb" + averagePositionVerb;
    root_pos += "_stringNoun" + averagePositionNoun;
    int right = Math.abs(averagePositionNoun -
averagePositionVerb);
1;
int position = (averagePositionNoun + averagePositionVerb) >>
if (!command_V.cartesianWorkActionsRightsV0.containsKey(root)
    && !command_V.cartesianWorkActionsPositionsV0
    .containsKey(root)

```



```
doMap.put("数据", true);
doMap.put("智慧", true);
doMap.put("逻辑", true);
doMap.put("选择", true);
doMap.put("操作", true);
doMap.put("点击", true);
doMap.put("点", true);
doMap.put("确认", true);
doMap.put("做", true);
doMap.put("将", true);
doMap.put("获取", true);
doMap.put("获", true);
doMap.put("获得", true);
doMap.put("取得", true);
doMap.put("取", true);
doMap.put("授权", true);
doMap.put("授", true);
doMap.put("选", true);
doMap.put("确定", true);
doMap.put("确保", true);
doMap.put("认准", true);
doMap.put("认", true);
doMap.put("认定", true);
doMap.put("定", true);
doMap.put("标记", true);
doMap.put("标出", true);
doMap.put("标", true);
doMap.put("拿出", true);
doMap.put("拿到", true);
doMap.put("拿来", true);
doMap.put("拿", true);
fixMap.put("把", true);
fixMap.put("被", true);
doMap.put("锁定", true);
doMap.put("锁存", true);
doMap.put("锁", true);
doMap.put("存", true);
// （进行 执行 跟进 更近 更进 数据 智慧 逻辑 选择 操作 确认）
}
```

```
package S_A.SixActionMap;
@SuppressWarnings("unchecked")
public class WorkVerbalMap extends WorkVerbalMap_X {
```

// 一些逻辑不应该出现在电脑上，只能文字出现在书本上。就因为电脑内置蓝牙wifi声卡接口


```
//, 我就不爽。不管了我就当写书一样就是了。--罗瑶光 trif
/*
* 1 华瑞集的所有函数通过元基花进行注册登记, 注册登记的函数进行24组十六元基编码索引分类
* 模拟染色体 可以序列化索引和编码所有函数。AOPM VECS IDUQ 稍后就能用上了。
*
* 2 函数调用接口通过元基枝进行归纳分类, 归纳分类的接口通过功能测试进行功能分类成六元分析机
* 模拟生活领域的生存技能。六元分析机 -爱-学习-帮助-创新-安全-工作-
*
* 3 通过笛卡尔关系计算将六元功能分析机 与 24组十六元基编码索引分类函数 进行指令集熵化结
果。
* 构造一个内存大脑tin map 保存这个熵化状态。
*
* 4 增加面向对象继承孢子形态 OSGI形态 花语形态 手工编码形态 测试函数融合 4种形态扩展插件
* 指令集和VPCS业务逻辑。
*
* 5 通过一次新陈代谢逻辑和二次新陈代谢逻辑来缩进函数名, 文件名, 标题名, 变量名, 宏名, 类名
* 增加计算效率, 肽展公式进行优化关系。
*
* 这上述5点的-DNA元基催化与肽计算-过程保证了以后华瑞集函数指令系统进行仿生拟态杂交仅仅融合
* 测试文件和测试文件的统计自然选择确定那些函数和指令在杂交后保留, 优先, 剔除, 等操作逻辑即可。
*
* 目前这个框架越来越规范和完善。离不开详细的文字描述。这是哲学的分支。慢慢开始意识到,
* 人类的进化过程中一个明显的导向是对时间事物的具体描述能力。这个能力也是认知能力。所以年轻
* 人一定要学好语文, 特别是其中的散文 叙述文和议论文。
*
* 写到这我开始思考笛卡尔关系的去重逻辑, 笛卡尔关系属于全耦合关系, 不符合量子计算的规范。
* 关于笛卡尔关系优化, 我想到了很多点,
* 1 首先便是去重, 可以构造一个临时map来辨别当前子关系是否被重复计算。定义
cartesianLooped
* 2 局部微分化, 将段落句拆分为单句集, 子单句拆分为 DNN词汇句。DNN词汇句做分词笛卡尔关系计
算。
* 3 多层笛卡尔关系做sigma熵化, 关系的过程变量进行微积分优化求导, 结果作加法累积。
* 4 稳定功能测试后进行 -DNA元基催化与肽计算- 上述5点算法融合。
* 5 AOPM VECS IDUQ 指令集索引 稍后就能用上了。
*
* 于是跟进这个5点小目标。
* 关于DNN的词汇归纳思考- 仅仅在子单句的长度超过30字-精度30可设成变量可自由设计-的条件下
* 触发DNN替代原句分析。避免繁琐计算, 达到有效计算加速的目的。
*
* --罗瑶光
*
* */
public boolean findSubject(App NE, CommandClass command_V) {
    initEnvironment();
    // small talk calculus
    // m 一旦笛卡尔, 单字组合就没有用了, 仅仅依赖分词即可。
    relationshipsCombinationWithNoun();
}
```

```

// d 看了计算哲学后，我才意识到我40年生命中语文功底算是白学了。
relationshipsCombinationWithVerb();
// md
relationshipsCombinationWithNounAndVerb();
/*
 * 发现很多变量在逻辑意识优化后，关系也变化了，一些计算产物之后都没有用到了
 * 于是先保留，方便之后的逻辑更近。
 * */
// init cartesianActions
initCartesianActions(NE, command_V);
//SV0的关系细化分解后，逻辑操作会更加地精确。
sortCartesianWorkActionsPositionSV(NE, command_V);
sortCartesianWorkActionsDistanceSV(NE, command_V);
sortCartesianWorkActionsPositionV0(NE, command_V);
sortCartesianWorkActionsDistanceV0(NE, command_V);
actionsNormalization(NE, command_V);
if (!objectMap.isEmpty() && !verbMap.isEmpty()) {
    return true;
}
return false;
}
/*
 * 新的商业测试接口既然写了那就要用。作为世界记录的7年保持者，我罗瑶光要做的 只有2件事情，1
 * 挑战我自己和期待对手挑战我，2 改变我自己对以往事物的评价， 给自己传道授业解惑。什么时候
 * 被业界打倒了，我就退休。好多东西等我玩。写代码只是我的兴趣爱好。别抽象我。我只是个凡人。
 */
@SuppressWarnings("unused")
public void setHumanTalkAfterNewBusinessTest(CommandClass command_V,
        App NE) {
    /*
     * 在进行分词前进行数字提取过滤，得到数字类nums和序次的map然后过滤掉这些数字
     * 的string进行下一步的操作，如果有alfs的提取任务，就alfs也用这个逻辑处理。
     * --罗瑶光
     * */
    int res = new StudyVerbalMap().extractNumberfromString(command_V);

    this.humanTalk = command_V.command;
    // 分词 提取 英文段和数字段形成变量。比如dnn 12345等
    System.out.println("chineseSimpleCommandWithoutNumerics400-->"
        + command_V.chineseSimpleCommandWithoutNumerics);
    command_V._IMV_SIQ_SS_ = NE.app_S._A
        .parserMixedString(command_V.chineseSimpleCommandWithoutNumerics);
    // -1 词频 归纳
    // -2 词性 归纳
    // 之前逻辑是 所有词性词汇 搜索 归纳
    // -未知 词汇 入 NE.app_S.workVerbalMap.unknown_map.put(string,
    // true);
    // 其他 入 mapSearchWithoutSort.put(string, wordFrequency);
    // 测试逻辑是 名 动 形 副 修正归纳 map-string-wordFrequency
    // 缺少逻辑是

```

```

// 增加其他词性 map 同时统一入 mapSearchWithoutSort 即可逻辑有很多种,
// 我选择 都做一遍, 然后loop 替换即可我的动机是确保包含所有形式的 完整计算关系。
//
command_V._IMV_SIQ_SS = NE.app_S._A
.getWordFrequencyMap(command_V._IMV_SIQ_SS_, NE);
NE.app_S._A.initPCAWordPOS(command_V._IMV_SIQ_SS, NE);
//
IMV_SIQ pos = NE.app_S._A.getPosCnToCn();
DemoPOSTest demoPOSTest = new DemoPOSTest();
//demoPOSTest.testPOS(command_V._IMV_SIQ_SS_, pos);
//--解决方法 首先分解
List<String> setsInput =
demoPOSTest.testPOSonlyGetList(command_V._IMV_SIQ_SS_, pos);
/*
    * 思考, 当一个原来的词汇关系系统计算中, 纠正了副词的准确性, 那么原来的函数中形容
词的词数
    * 就会大幅地减少, 如果之后的跟进计算用到了形容词, 而没有用到副词, 那么条件的精度
会增加,
    * 而过滤数也会增加。这样的计算强调语法包含, 质量提高, 但性能降低。提高性能的方式
是增加

    * 副词逻辑的计算函数集。--罗瑶光
    */
/*
    * 在测试文档中已经设计了更为精准的 DemoAfterPOSTest 服务, 于是这里进行替换下函
数, 看效果如何。

    * testAfterPOS 的逻辑是 出现单字的 同词性相联的进行合并, 但是这个格式在当前的
tinshell逻辑中

    * 不能采纳, 于是需要稍微的变形。编程可采纳的格式。
    *
    * 问题1 demoPOSTest.testPOS 已经将改变的词性修正 到wordFrequency 类中还注
册了

    * char position 直接变形比较麻烦, 于是开始梳理思路。
    * --解决方法 首先分解 demoPOSTest.testPOS 为两个逻辑, 先修正pos 变成 含有/
的 list

    * --再list组合修正。--最终将修正的list 处理成 wordFrequency 格式list。
    *
    * */
//--再list组合修正
DemoAfterPOSTest demoAfterPOSTest = new DemoAfterPOSTest();
List<String> setsAfterInput =
demoAfterPOSTest.testAfterPOS(setInput, pos);
//--最终将修正的list 处理成 wordFrequency 格式list。
demoPOSTest.testPOStoWordFrequencyList(setAfterInput, pos);
// loop update and insect
Iterator<String> iterators = demoPOSTest.noun.keySet().iterator();
while (iterators.hasNext()) {
    String temp = iterators.next();
    if (command_V._IMV_SIQ_SS.containsKey(temp)) {
        WordFrequency wordFrequency = command_V._IMV_SIQ_SS.get(temp);

```

```

WordFrequency wordFrequencyTemp = demoPOSTest.noun.get(temp);
wordFrequency.I_pos(wordFrequencyTemp.get_pos());
command_V._IMV_SIQ_SS.put(temp, wordFrequency);
    }
}
iterators = demoPOSTest.verb.keySet().iterator();
while (iterators.hasNext()) {
    String temp = iterators.next();
    if (command_V._IMV_SIQ_SS.containsKey(temp)) {
WordFrequency wordFrequency = command_V._IMV_SIQ_SS.get(temp);
WordFrequency wordFrequencyTemp = demoPOSTest.verb.get(temp);
wordFrequency.I_pos(wordFrequencyTemp.get_pos());
command_V._IMV_SIQ_SS.put(temp, wordFrequency);
    }
}
iterators = demoPOSTest.adj.keySet().iterator();
while (iterators.hasNext()) {
    String temp = iterators.next();
    if (command_V._IMV_SIQ_SS.containsKey(temp)) {
WordFrequency wordFrequency = command_V._IMV_SIQ_SS.get(temp);
WordFrequency wordFrequencyTemp = demoPOSTest.adj.get(temp);
wordFrequency.I_pos(wordFrequencyTemp.get_pos());
command_V._IMV_SIQ_SS.put(temp, wordFrequency);
    }
}
iterators = demoPOSTest.adv.keySet().iterator();
while (iterators.hasNext()) {
    String temp = iterators.next();
    if (command_V._IMV_SIQ_SS.containsKey(temp)) {
WordFrequency wordFrequency = command_V._IMV_SIQ_SS.get(temp);
WordFrequency wordFrequencyTemp = demoPOSTest.adv.get(temp);
wordFrequency.I_pos(wordFrequencyTemp.get_pos());
command_V._IMV_SIQ_SS.put(temp, wordFrequency);
    }
}
}

public void setHumanTalk(CommandClass command_V, App NE) {
    command_V._IMV_SIQ_SS.clear();
    command_V._IMV_SIQ_SS_.clear();
    command_V._IMV_SIQ_S_.clear();
    this.humanTalk = command_V.command;
    // 分词 提取 英文段和数字段形成变量。比如dnn 12345等
    command_V._IMV_SIQ_SS_ = NE.app_S._A
        .parserMixedString(command_V.command);
    for (int i = 0; i < command_V._IMV_SIQ_SS_.size(); i++) {
        System.out.println(command_V._IMV_SIQ_SS_.get(i));
    }
    // 1 精确词汇pos函数
    // 2 精确词汇笛卡尔 取缔之前的老快速 map 频率
    command_V._IMV_SIQ_SS = NE.app_S._A
        .getWordFrequencyMap(command_V._IMV_SIQ_SS_, NE);
    // 3 精确词汇rnn 和 position

```

```

        // loop unknown
        // 4 精确词汇的mapping肽指令集
        // 5 局部替换即可，价值可识别12345和英文abcde 方便人类语言中入参识别。
        // */
        NE.app_S._A.initPCAWordPOS(command_V._IMV_SIQ_SS, NE);
    }

    // 先处理仅一个主谓宾的简单长句，以后处理复杂带连词的多主宾复句子。
    // 在输出的数据表中仅展示列名为中药名称，打分和功效列这三个即可
    /*
    * 这个把被逻辑以后用指令集的形式分出去。--later
    * */
    public String returnBestTypeOfCommands(Boolean findSubject) {
        // init shortChineseActions
        Iterator<String> iterator = objectMap.keySet().iterator();
        while (iterator.hasNext()) {
            String subject = iterator.next();
            if (babeiMap.containsKey("把")) {
                if (objectMap.getInt(subject) < babeiMap.getInt("把")) {
                    subjectName += subject;
                }
                if (objectMap.getInt(subject) > babeiMap.getInt("把")) {
                    objectName += subject;
                }
            } else if (babeiMap.containsKey("被")) {
                if (objectMap.getInt(subject) < babeiMap.getInt("被")) {
                    objectName += subject;
                }
                if (objectMap.getInt(subject) > babeiMap.getInt("被")) {
                    subjectName += subject;
                }
            } else {
                if (null == subjectName) {
                    subjectName += subject;
                } else {
                    objectName += subject;
                }
            }
        }
        if (findSubject) {
            String output = "";
            if (null != subjectName) {
                output = subjectName;
            }
            output += ":";
            if (null != doName) {
                output += doName;
            }
            output += ":";
            if (null != objectName) {
                output += objectName;
            }
        }
    }

```

```

        return output;// later
    }
    return null;
}
}
//计算辩证意识大幅减少大量算子。
##### 2
##### 混合中文数字进行识别对应阿拉伯字符在Tinshell中的具体应用和优化方式 --源码如下

```

```

-----
package S_A.SixActionMap;
public class StudyVerbalMap_Q extends StudyVerbalMap_X {
    /*
    * System.out.println("混合数字字符预处理锁定-->" + string);
    * 因为有些大佬喜欢写100万2000这种标识，就不用一百万两千和1002000这类规范的。
    * 所以我在这个if里面之后还要设计个阿拉伯数字转汉字的数字翻译机。 逻辑是先拆分数汉，
    * 再翻译数变汉，最后组合全汉输出即可。 --罗瑶光
    *
    * 首先构造一个getChineseFromNumerics，我设置最大数为16位，因为之前的
    * getNumericsFromChinese也是16位，我电脑最大也就8位。大数运算我不cover。
    */
    @SuppressWarnings("unused")
    public String getChineseFromNumerics(String number) {
        System.out.println(number);
        /*
        * 首先开始思考，假定这个number是一个标准的整数，因为小数逻辑简单，
        * 只要直接翻译char即可。负数只要前面价格符号即可。
        *
        * 于是开始分解，4位的千万位一个区间，那么是4个区间，符合普通人理解。
        * numberParser[0]=0~9999
        * numberParser[1]=0 0000~9999 0000
        * numberParser[2]=0 0000 0000~9999 9999 9999
        * numberParser[3]=0 0000 0000 0000~9999 9999 9999 9999
        * 这就简单了，直接加个戳组合即可。
        * --论证了 计算哲学 适合 所有定义类逻辑的描述。
        * --罗瑶光
        */
        char[] chars = number.toCharArray();
        String[] chineseParser = new String[4];
        String[] chineseParsered = new String[4];
        for (int i = 0; i < chineseParser.length; i++) {
            chineseParser[i] = "";
            chineseParsered[i] = "";
        }
        /*
        * 1 number swap to numberParser
        */
        int order = 0;
        for (int right = chars.length - 1; right >= 0; right--) {
            char temp = chars[right];
            if (order < 4) {
                chineseParser[0] = temp + chineseParser[0];
            }
        }
    }
}

```

```

        } else if (order < 8) {
chineseParser[1] = temp + chineseParser[1];
        } else if (order < 12) {
chineseParser[2] = temp + chineseParser[2];
        } else if (order < 16) {
chineseParser[3] = temp + chineseParser[3];
        }
        order++;
    }
    for (int i = 0; i < chineseParser.length; i++) {
        // System.out.println(i + "-->" + chineseParser[i]);
    }
    /*
    * 2 numberParser swap to chineseParser
    */
    for (int i = 0; i < chineseParser.length; i++) {
        chineseParsered[i] =
doSwapUnderTenThousands(chineseParser[i]);
        // System.out.println(i + "-->" + chineseParsered[i]);
    }
    /*
    * 3 chineseParser combination
    */
    String output = "";
    boolean has3 = false;
    boolean has2 = false;
    boolean has1 = false;
    boolean has0 = false;
    if (chineseParsered[3].isEmpty()) {
        has3 = false;
    } else {
        has3 = true;
    }
    output = output + chineseParsered[3];
    if (true == has3 && !chineseParsered[3].equals("零")) {
        output = output + '万';
    }
    //
    if (chineseParsered[2].isEmpty()) {
        has2 = false;
    } else {
        has2 = true;
    }
    output = output + chineseParsered[2];
    if ((true == has2 || true == has3) && !
chineseParsered[2].equals("零")) {
        output = output + '亿';
    }
    //
    if (chineseParsered[1].isEmpty()) {
        has1 = false;
    } else {
        has1 = true;
    }

```

```

        output = output + chineseParsered[1];
        if ((true == has1) && !chineseParsered[1].equals("零")) {
            output = output + '万';
        }
        //
        if (chineseParsered[0].isEmpty()) {
            has0 = false;
        } else {
            has0 = true;
        }
        output = output + chineseParsered[0];
        /*
        * 开头与结尾零过滤,因为正则在不同的系统中有不同的语法格式如PCRE
        * 所以java系统我手写一份。 --罗瑶光
        */
        //String outputFinal = "";
        output = prefixOptimization(output);
        // oder-fix
        output = orderfixOptimization(output);
        System.out.println("output-->" + output);
        return output;
    }

    /*
    * fix filter 和 oder-fix稍后提取成函数,避免重复,然后command class
    * 继承这些中间变量, 处理好哲学关系 保持简洁计算性能。。 --罗瑶光
    */
    public String prefixOptimization(String input) {
        if (!input.isEmpty()) {
            while (input.charAt(0) == '零' && input.length() > 1) {
                input = input.substring(1, input.length());
            }
        }
        return input;
    }

    public String orderfixOptimization(String input) {
        if (!input.isEmpty()) {
            while (input.charAt(input.length() - 1) == '零'
                && input.length() > 1) {
                input = input.substring(0, input.length() - 1);
            }
        }
        return input;
    }

    /*
    * 这个逻辑开始思考千位变换, 1- 首先string 进行 swap to chars
    * 2- 4位的 char 对应个十百千 这是单位, 3- char是0-9 需要量词翻译。
    * 4- 翻译后加单位进行组合, 量词是零需要过滤所在的单位, 因为一开始是高位满足法拆分,
    * 5- 所以开头是0, 也要保留零,
    * 6- 末尾是零需要过滤,

```



```
*/
@SuppressWarnings("unused")
public String doSwapUnderTenThousands(String string) {
    String stringChinese = "";
    String stringChineseUnits = "";
    String stringChineseUnitsFix = "";
    String stringChineseUnitsFixFlter = "";
    String output = "";
    char[] chars = string.toCharArray();
    for (int right = chars.length - 1; right >= 0; right--) {
        char temp = chars[right];
        if (temp == '0') {
            stringChinese = "零" + stringChinese;
        }
        if (temp == '1') {
            stringChinese = "一" + stringChinese;
        }
        if (temp == '2') {
            stringChinese = "二" + stringChinese;
        }
        if (temp == '3') {
            stringChinese = "三" + stringChinese;
        }
        if (temp == '4') {
            stringChinese = "四" + stringChinese;
        }
        if (temp == '5') {
            stringChinese = "五" + stringChinese;
        }
        if (temp == '6') {
            stringChinese = "六" + stringChinese;
        }
        if (temp == '7') {
            stringChinese = "七" + stringChinese;
        }
        if (temp == '8') {
            stringChinese = "八" + stringChinese;
        }
        if (temp == '9') {
            stringChinese = "九" + stringChinese;
        }
    }
    // System.out.println("stringChinese-->" + stringChinese);
    /*
    * 加单元
    */
    int order = 0;
    for (int i = stringChinese.length() - 1; i >= 0; i--) {
        stringChineseUnits = "" + stringChinese.charAt(i);
        // System.out.println("stringChineseUnits-->" +
        // stringChineseUnits);
        if (1 == order && stringChinese.charAt(i) != '零') {
```

```

        stringChineseUnits = stringChinese.charAt(i) + "+";
        // System.out
        // .println("stringChineseUnits-->" + stringChineseUnits);
    }
    if (2 == order && stringChinese.charAt(i) != '零') {
        stringChineseUnits = stringChinese.charAt(i) + "百";
        // System.out
        // .println("stringChineseUnits-->" + stringChineseUnits);
    }
    if (3 == order && stringChinese.charAt(i) != '零') {
        stringChineseUnits = stringChinese.charAt(i) + "千";
        // System.out
        // .println("stringChineseUnits-->" + stringChineseUnits);
    }
    stringChineseUnitsFix = stringChineseUnits +
stringChineseUnitsFix;
    order++;
}
// System.out.println("stringChineseUnitsFix-->" +
// stringChineseUnitsFix);
/*
 * 过滤零单元
 */
stringChineseUnitsFix = stringChineseUnitsFix.replace("零零零零", "");
stringChineseUnitsFix = stringChineseUnitsFix.replace("零零零", "零");
stringChineseUnitsFix = stringChineseUnitsFix.replace("零零", "零");
// System.out.println("stringChineseUnitsFixFilter-->" +
// stringChineseUnitsFix);
// pre-fix
// oder-fix
stringChineseUnitsFix = orderfixOptimization(stringChineseUnitsFix);
return stringChineseUnitsFix;
}

public static void main(String[] argv) {
}
}

```

```

package S_A.SixActionMap;
public class StudyVerbalMap extends StudyVerbalMap_Q {
    public String filterString = "";
    /*
     * 这个函数用于command_V的commandString中提取出数字特征字符到map中，然后遍历map
     * 将这些特征字符的词组进行过滤掉。
     */
    public int extractNumberfromString(CommandClass command_V) {
        if (null == command_V.command) {
            return -1;
        }
        if (null == command_V.command) {
            return -1;
        }
    }
}

```

```

        // command_V.getNumericsFromUnknownMap(command_V.command);
        getNumericsFromUnknownMapAndFiltCommand(command_V);
        return 0;
    }

    /*
    * 这个函数用于将command string中的数字和汉字进行提取出来，形成一个map，如果是汉字就进行
    * 阿拉伯数字变换。提取map后将原string的对应字符全部过滤掉生成commandWithNumFilters。
    *
    */
    @SuppressWarnings("unchecked")
    public void getNumericsFromUnknownMapAndFiltCommand(
        CommandClass command_V) {
        // TODO Auto-generated method stub
        // number extra
        String string = "";
        String inputString = command_V.command;
        inputString = command_V.simpleChineseNumberSwap(inputString);
        /*
        * 思考--涉及拆嵌套的逻辑如何分关系。
        */
        if(command_V.chineseSimpleCommandWithoutNumerics.isEmpty()) {
            command_V.chineseSimpleCommandWithoutNumerics =
inputString.toString();
        }
        System.out.println("chineseSimpleCommandWithoutNumerics400-1-->"
+ command_V.chineseSimpleCommandWithoutNumerics);
        int fixOrder = 0;
        for (fixOrder = 0; fixOrder < inputString.length(); fixOrder++) {
            if ((inputString.charAt(fixOrder) > 47
&& inputString.charAt(fixOrder) < 58)
|| (inputString.charAt(fixOrder) == '零'
|| inputString.charAt(fixOrder) == '一'
|| inputString.charAt(fixOrder) == '二'
|| inputString.charAt(fixOrder) == '三'
|| inputString.charAt(fixOrder) == '四'
|| inputString.charAt(fixOrder) == '五'
|| inputString.charAt(fixOrder) == '六'
|| inputString.charAt(fixOrder) == '七'
|| inputString.charAt(fixOrder) == '八'
|| inputString.charAt(fixOrder) == '九'
|| inputString.charAt(fixOrder) == '十'
|| inputString.charAt(fixOrder) == '百'
|| inputString.charAt(fixOrder) == '千'
|| inputString.charAt(fixOrder) == '万'
|| inputString.charAt(fixOrder) == '亿'
|| inputString.charAt(fixOrder) == '壹'
|| inputString.charAt(fixOrder) == '贰'
|| inputString.charAt(fixOrder) == '两'
|| inputString.charAt(fixOrder) == '叁'

```

API

API

char position

```

|| inputString.charAt(fixOrder) == '肆'
|| inputString.charAt(fixOrder) == '伍'
|| inputString.charAt(fixOrder) == '陆'
|| inputString.charAt(fixOrder) == '柒'
|| inputString.charAt(fixOrder) == '捌'
|| inputString.charAt(fixOrder) == '玖'
|| inputString.charAt(fixOrder) == '拾'
|| inputString.charAt(fixOrder) == '佰'
|| inputString.charAt(fixOrder) == '仟')) {
    } else {
if (!string.isEmpty()) {
    System.out.println(string + "--" + fixOrder);
    // println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测

    command_V.numericsFromUnknownString.put(string.toString(),
        fixOrder);
    string = "";
}
filterString += inputString.charAt(fixOrder);
continue;
}
string += inputString.charAt(fixOrder);
}
if (!string.isEmpty()) {
    System.out.println(string + "--" + fixOrder);
    // println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测

    command_V.numericsFromUnknownString.put(string.toString(),
        fixOrder);
    string = "";
}
// chinese number extra
/*
 * 之后这类输出统一归纳为频率 和时间统计函数集，方便高频优先意识。 --罗瑶光
 */
// english extra later
}

@SuppressWarnings("unchecked")
public void formatNumericMap(CommandClass command_V) {
    Iterator<String> iterators = command_V.numericsFromUnknownString
        .keySet().iterator();
    while (iterators.hasNext()) {
        String string = iterators.next();
        String symbolsSwapNumericsByLength = "";
        for(int i = 0; i < string.length(); i++) {
symbolsSwapNumericsByLength += command_V.symbolSwapNumerics;
        }
        /*
         * symbolsSwapNumericsByLength 与string的 长度一致 增加分词的
char position
         * 准确度，因果增加笛卡尔关系的精确度。

```

```

        * --罗瑶光
        * */

        System.out.println("混合数字字符探索-->" + string);
        boolean hasNumerics = false;
        boolean hasChars = false;
        if (string.contains("0") || string.contains("1")
            || string.contains("2") || string.contains("3")
            || string.contains("4") || string.contains("5")
            || string.contains("6") || string.contains("7")
            || string.contains("8") || string.contains("9")) {
hasNumerics = true;
        }
        if (string.contains("零") || string.contains("一")
            || string.contains("二") || string.contains("三")
            || string.contains("四") || string.contains("五")
            || string.contains("六") || string.contains("七")
            || string.contains("八") || string.contains("九")
            || string.contains("十") || string.contains("十")
            || string.contains("百") || string.contains("千")
            || string.contains("万") || string.contains("亿")) {
hasChars = true;
        }
        if (hasNumerics && !hasChars) {
/*
    * 翻译阿拉伯数字 逻辑是
    * number = studyVerbalMap_Q.getChineseFromNumerics(number);
    * */
String stringSwaped = getChineseFromNumerics(string);
System.out.println("stringSwaped-400-1->" + stringSwaped);
stringSwaped = command_V.fasterChineseNumberSwap(stringSwaped);
System.out.println("stringSwaped-400-2->" + stringSwaped);
//String stringSwaped = command_V.fasterChineseNumberSwap(string);
WordFrequency wordFrequency = new WordFrequency(1, stringSwaped);
String temp = command_V.numericsFromUnknownString.getString(string);
int tempInt = Integer.valueOf(temp);
System.out.println("position-->" + tempInt);
wordFrequency.positions.add(tempInt);
//wordFrequency.I_frequency(1);
wordFrequency.I_pos("变换数字字符串代词名词");
command_V._IMV_SIQ_SS_Q.put(stringSwaped, wordFrequency);
command_V.chineseSimpleCommandWithoutNumerics
= command_V.chineseSimpleCommandWithoutNumerics.replace(string
    , symbolsSwapNumericsByLength);
        }
        if (!hasNumerics && hasChars) {
/*
    * 翻译汉字数字 逻辑是
    * commandClass.fasterChineseNumberSwap(number);
    * 之后要归纳下, 避免重含, 如果有逆向递归操作容易死循环。
    * */
String stringSwaped = command_V.fasterChineseNumberSwap(string);

```

```

System.out.println("stringSwaped-400-2->" + stringSwaped);
WordFrequency wordFrequency = new WordFrequency(1, stringSwaped);
String temp = command_V.numericsFromUnknownString.getString(string);
int tempInt = Integer.valueOf(temp);
System.out.println("position-->" + tempInt);
wordFrequency.positions.add(tempInt);
//wordFrequency.I_frequency(1);
wordFrequency.I_pos("变换数字字符串代词名词");
command_V._IMV_SIQ_SS_Q.put(stringSwaped, wordFrequency);
command_V.chineseSimpleCommandWithoutNumerics
= command_V.chineseSimpleCommandWithoutNumerics.replace(string
    , symbolsSwapNumericsByLength);
/*
    * 下面注释的逻辑需要按照字符串的长短排序后按照长优先进行replace，不然会字符串断
层错误。
    */
    }
    if (hasNumerics && hasChars) {
System.out.println("混合数字字符预处理锁定-->" + string);
/*
    * 因为有些大佬喜欢写100万2000这种标识，就不用一百万两千和1002000这类规范的。
    * 所以我在这个if里面之后还要设计个阿拉伯数字转汉字的数字翻译机。 逻辑是先拆分数
    汉,
    * 再翻译数变汉，最后组合全汉输出即可。 --罗瑶光
    */
/*
    * loop 找出阿拉伯数字，然后翻译拼接
    */
String chineseNumber = "";
String arabicNumber = "";
for (int i = 0; i < string.length(); i++) {
    if (string.charAt(i) > 47 && string.charAt(i) < 58) {
        arabicNumber += string.charAt(i);
    } else {
        if (!arabicNumber.isEmpty()) {
            String chineseNumberPars =
getChineseFromNumerics(
                arabicNumber);
            String fix = "";
            System.out.println(
                "chineseNumber-->" + chineseNumber);
            System.out.println("chineseNumberPars
length-->"
                + chineseNumberPars.length());
            if (!chineseNumber.isEmpty()
                && arabicNumber.length() < 4) {
                fix = "零";
            }
            chineseNumber += fix + chineseNumberPars;
            arabicNumber = "";
        }
        chineseNumber += string.charAt(i);

```

```

        }
    }
    if (!arabicNumber.isEmpty()) {
        String chineseNumberPars = getChineseFromNumerics(
            arabicNumber);
        String fix = "";
        System.out.println("chineseNumber-->" + chineseNumber);
        System.out.println("chineseNumberPars length-->"
            + chineseNumberPars.length());
        if (!chineseNumber.isEmpty() && arabicNumber.length() < 4) {
            fix = "零";
        }
        chineseNumber += fix + chineseNumberPars;
        arabicNumber = "";
    }
    // fix filter
    chineseNumber = prefixOptimization(chineseNumber);
    // oder-fix
    chineseNumber = orderfixOptimization(chineseNumber);
    System.out.println("混合数字字符预处理结果-->" + chineseNumber);
    //稍后去重
    /*
     * 思考关于position的位置添加方式，是字符串的头字所取位置，那么之后分词的
     * 位置也要符合这个规范，不然有误差。或导致精度过滤的条件类计算不准确。
     * later -trif。 罗瑶光
     */

    String regArabicNumber =
command_V.fasterChineseNumberSwap(chineseNumber);
    System.out.println("stringSwaped-400-2->" + regArabicNumber);
    WordFrequency wordFrequency = new WordFrequency(1, regArabicNumber);
    String temp = command_V.numericsFromUnknownString.getString(string);
    int tempInt = Integer.valueOf(temp);
    System.out.println("position-->" + tempInt);
    wordFrequency.positions.add(tempInt);
    //wordFrequency.I_frequency(1);
    wordFrequency.I_pos("变换数字字符串代词名词");
    command_V._IMV_SIQ_SS_Q.put(regArabicNumber, wordFrequency);
    command_V.chineseSimpleCommandWithoutNumerics
= command_V.chineseSimpleCommandWithoutNumerics.replace(string
        , symbolsSwapNumericsByLength);
    }
}

    public static void main(String[] argv) {
}
/*
 * 病句歧义分析123万200亿000 也可以理解为一百二十三万零二百亿仟，但是这里出了一个问题， 如果后面接
123,
 * 如123万200亿123, 那么这里有两个意思，1是一百二十三万零二百亿 个 123,

```

```

* 要做乘积，乘123，在这种计算逻辑环境中，数字提取的将不是稳定的数字提取逻辑，而是数字乘法
* 函数的提取，提取的是一个要计算乘法的逻辑。可以理解为 病句歧义分析含有算法逻辑的数字函数。
* 所以不考虑，按完整数字逻辑来获取。按拼接法 123万200亿000 需要改为 123万200亿， 再举
* 个例子123万200亿2000则保留末尾2000，因为2字有数字拼接逻辑意义。 --罗瑶光
*
*/

```

其他都是元基编码的原函数，原著思想的测试组合和优化修正，这里全部略，
只有这个TinShell中的笛卡尔指令词汇关系和阿拉伯数字识别属于新加创作，先申请著作权。

对了分词的切词后组词逻辑也申请下著作权，增加分词精确度的逻辑，改写了业界的分词评审逻辑。

```
package test.java.InterfaceTest.chineseParser;
```

```

/*
* 关于定制map的思考
* 并不是当前世界的所有的领域都要特地专门地设计一些定制map来进行组词，因为定制map主要用来
* 处理稀奇古怪的名词缩写问题。对于变化快速的街道，传媒，食品等某些领域特殊现象归纳，当这个潮流一旦过了
* 热度没了，这些新兴名词之后也会随着时间莫名消失，添加的方式应该是首先扩充比较通用的名词词汇。
* 所以这些定制词汇应该需要一个频率属性来统计使用次数，或者使用前和当前的官方开源词库进行比对下，
* 如交通街道名这类全世界几百万个街道，命名毫无逻辑，想改就能改的组合街名词。
* 过滤的价值在于提高组词识别效率。
* */
public class ParserCharsFix {
    // 索引词汇定制map是固定变量表，可以全局static使用。
    public static Map<String, HashMap<String, String>> environmentIndex = new
HashMap<>();

    public void initenvironmentIndex(App NE) throws IOException {
        initMilitary(NE);
        initMeeting(NE);
        initNation(NE);
        initMusic(NE);
        initCarFix(NE);
        initFood(NE);
        initCity(NE);
        initStreet(NE);
        // ... .
    }
    public void initMilitary(App NE) throws IOException {
        /*
        * 词汇越来越多之后可以lyg文件进行文件init扩展。
        */
        HashMap<String, String> military = new HashMap<>();
        callFastReadProjectFile(military, "military.lyg", NE, "otherlyg"
, S_Pos.UTF8_STRING, "/");
        System.out.println("400-0000000001-" + military.size());
        environmentIndex.put("军事", military);
        // 还可以细化
    }
}

```



```

    }
    public void callFastReadProjectFile(HashMap<String, String> inputs
        , String fileName, App NE, String category, String charSet
        , String lineParserSymbol) throws IOException {
        /*
        * 问题, 当时设计读取资源函数, 没有细致到设计成一种非常方便记忆的函数格式。
        * 导致每次使用我还要想半天这个逻辑, 怎么扩展和重用。
        *
        * --罗瑶光
        */
        DetabufferedReader cReaderp =
FastReadProjectFile.getDetabufferedReader(
        fileName, NE.resourceTail + category + "/", charSet);
        // index
        String cInputStringp;
        while ((cInputStringp = cReaderp.readDetaLine()) != null) {
            String[] words = cInputStringp.split(lineParserSymbol);
            if (words.length > 1) {
                inputs.put(words[0], "可英文");
            }
        }
        cReaderp.close();
    }

    public void initMeeting(App NE) throws IOException {
        HashMap<String, String> meeting = new HashMap<>();
        callFastReadProjectFile(meeting, "meeting.lyg", NE, "otherlyg"
        , S_Pos.UTF8_STRING, "/");
        System.out.println("400-0000000002-" + meeting.size());
        environmentIndex.put("会议", meeting);
    }

    public void initNation(App NE) throws IOException {
        HashMap<String, String> nation = new HashMap<>();
        callFastReadProjectFile(nation, "nation.lyg", NE, "otherlyg"
        , S_Pos.UTF8_STRING, "/");
        System.out.println("400-0000000003-" + nation.size());
        environmentIndex.put("国家", nation);
    }

    //...

    /*
    * --通过德塔极速切词后的结果开始获取联想环境词汇map, 之前很多大佬说我的
    * LenovoInit函数没用, 搞了干嘛, 我本想认真解释 既然没用那你们找我干嘛,
    * 后来想想算了, 本就互不认识, 争论无意义, 现在做定制map校正和输出校正测试
    * , 我就写点文字来描述下LenovoInit一些2019年的具体的逻辑价值。
    * --关于环境的定制map初始有多种方式, 我介绍2种 -1 数据分词时候已经知道了
    * 具体的环境类, 可以在分词前进行init针对需要遍历的环境赋值= “历史, 哲学 , 数学。。。”
    *
    * 数据分词时不知道具体, 那么就需要用lenovoInit来 获取大概环境类如 “历史, 哲学, 数
    * 学。。。”
    */

```

-2

```

    * 然后进行遍历组字检查。 前提条件是lenovo map需要扩充细致完整。
    * --罗瑶光
    */
public Map<String, String> getInvironment(App NE, List<String> sets) {
    Map<String, String> invironmentMap = new HashMap<>();
    // 稍后函数提取出来, 小片段化去重。
    NE.app_S.lenovoInit.IV_SetsExclude_A(sets, NE);
    IMV_SIQ_X_environmentSampleMap = NE.app_S.lenovoInit
        .getEnvironmentInit().getEmotionSampleMap();
    IMV_SIQ lenovo = NE.app_S.lenovoInit.getSensingMap().getLenovoMap();
    // reduce
    System.out.println("环    境: ");
    Iterator<String> Iterator = environmentSampleMap.keySet().iterator();
    while (Iterator.hasNext()) {
        String word = Iterator.next();
        NE.app_S.emotionSample = environmentSampleMap.get_S(word);
        String stringDistinction =
NE.app_S.emotionSample.getDistinction();
        if (null != stringDistinction) {
            String string = "";
            if (lenovo.containsKey(stringDistinction)) {
                string = lenovo.get(stringDistinction).toString();
                invironmentMap.put(string.replace(" ", ""), "later");
            } else {
                string = NE.app_S.emotionSample.getDistinction();
                invironmentMap.put(string.replace(" ", ""), "later");
            }
            if (!string.replace(" ", "").isEmpty()) {
                System.out.print(string + " ");
            }
        }
    }
    return invironmentMap;
}
/*
    * 通过德塔极速切词后的结果开始获取联想环境词汇map校正最终结果。
    */
public List<String> charFix(App NE, List<String> sets) {
    List<String> computeSet = new LinkedList<>();
    List<String> subSet = new LinkedList<>();
    Iterator<String> iterators = sets.iterator();
    while (iterators.hasNext()) {
        String string = iterators.next();
        computeSet.add(string);
    }

    // 获取环境的组字涉及面。
    Map<String, String> invironmentMap = getInvironment(NE, sets);
    Iterator<String> iteratorsInvironmentMap = invironmentMap.keySet()
        .iterator();
    // 每一个环境面遍历
    while (iteratorsInvironmentMap.hasNext()) {
        String temp = iteratorsInvironmentMap.next();
    }
}

```

```

        // 如果词库的环境面恰好也在计算环境面, 就取出来计算当前组字面
        if (ParserCharsFix.environmentIndex.containsKey(temp)) {
Map<String, String> environmentMap = ParserCharsFix.environmentIndex
        .get(temp);
// 当前组字面从sets的 9字连词开始, 2个字结束
for (int i = 9; i > 1; i--) {
    // 逐词排查连字条件分析
    // 减少computeSet算子数
    // System.out.println("400-" + i);
    // System.out.println("");
    if (!subSet.isEmpty()) {
        computeSet.clear();
        Iterator<String> iteratorsSubset = subSet.iterator();
        while (iteratorsSubset.hasNext()) {
            String string = iteratorsSubset.next();
            // System.out.print(string + "-");
            computeSet.add(string);
        }
        subSet.clear();
    }
    System.out.println(" ");
    for (int j = 0; j < computeSet.size(); j++) {
        StringBuilder stringBuilder = new StringBuilder();
        boolean find = false;
        // 连字条件stringBuilder内核记录
        for (int k = j; k < computeSet.size()
            && k < j + i; k++) {
            if (stringBuilder.length() < i) {
                if (stringBuilder.length()
                    + computeSet.get(k).length() < i) {
                    stringBuilder.append(computeSet.get(k));
                }
            }
            String string = stringBuilder.toString();
            if (environmentMap.containsKey(string)) {
                // System.out.println("400-2-" + string);
                subSet.add(string);
                find = true;
                // 同时原list去掉string。
                stringBuilder.delete(0,
                    stringBuilder.length() - 1);
            }
            //
            j += (k - j);
        }
    }
    if (false == find) {
        // 如果没有找到就加原来的set, 如果找到了, 就要记录k
        // 如果k值是右交界点而substring处理了词中前字拆分逻辑,
        // 那么不考虑这类逻辑,
        // System.out.println("400-3-" +
        computeSet.get(j));
    }
}
}

```

```

        subSet.add(computeSet.get(j));
    }
    }
}
    }
}
if (!subSet.isEmpty()) {
    return subSet;
}
return sets;
}
}
```